



 Latest updates: <https://dl.acm.org/doi/10.1145/3547647>

RESEARCH-ARTICLE

Aeneas: Rust verification by functional translation

SON HO, INRIA Institut National de Recherche en Informatique et en Automatique, Le Chesnay, Ile-de-France, France

JONATHAN PROTZENKO, Microsoft Research, Redmond, WA, United States

Open Access Support provided by:

INRIA Institut National de Recherche en Informatique et en Automatique

Microsoft Research



PDF Download
3547647.pdf
02 April 2026
Total Citations: 41
Total Downloads:
3530



Published: 31 August 2022

[Citation in BibTeX format](#)



AENEAS: Rust Verification by Functional Translation

SON HO, Inria, France

JONATHAN PROTZENKO, Microsoft Research, USA

We present AENEAS, a new verification toolchain for Rust programs based on a lightweight functional translation. We leverage Rust's rich region-based type system to eliminate memory reasoning for a large class of Rust programs, as long as they do not rely on interior mutability or unsafe code. Doing so, we relieve the proof engineer of the burden of memory-based reasoning, allowing them to instead focus on *functional* properties of their code.

The first contribution of AENEAS is a new approach to borrows and controlled aliasing. We propose a pure, functional semantics for LLBC, a Low-Level Borrow Calculus that captures a large subset of Rust programs. Our semantics is value-based, meaning there is no notion of memory, addresses or pointer arithmetic. Our semantics is also ownership-centric, meaning that we enforce soundness of borrows via a semantic criterion based on *loans* rather than through a syntactic type-based *lifetime* discipline. We claim that our semantics captures the *essence* of the borrow mechanism rather than its current implementation in the Rust compiler.

The second contribution of AENEAS is a translation from LLBC to a pure lambda-calculus. This allows the user to reason about the original Rust program through the theorem prover of their choice, and fulfills our promise of enabling lightweight verification of Rust programs. To deal with the well-known technical difficulty of terminating a borrow, we rely on a novel approach, in which we approximate the borrow graph in the presence of function calls. This in turn allows us to perform the translation using a new technical device called *backward functions*.

We implement our toolchain in a mixture of Rust and OCaml; our chief case study is a low-level, resizing hash table, for which we prove functional correctness, the first such result in Rust. Our evaluation shows significant gains of verification productivity for the programmer. This paper therefore establishes a new point in the design space of Rust verification toolchains, one that aims to verify Rust programs simply, and at scale.

Rust goes to great lengths to enforce static control of aliasing; the proof engineer should not waste any time on memory reasoning when so much already comes “for free”!

CCS Concepts: • **Theory of computation** → **Programming logic; Logic and verification.**

Additional Key Words and Phrases: Rust, verification, functional translation

ACM Reference Format:

Son Ho and Jonathan Protzenko. 2022. AENEAS: Rust Verification by Functional Translation. *Proc. ACM Program. Lang.* 6, ICFP, Article 116 (August 2022), 31 pages. <https://doi.org/10.1145/3547647>

1 INTRODUCTION

In 2006, exasperated by yet another crash of his building's elevator's firmware, and exhausted after walking up 21 flights of stairs, Graydon Hoare set out to design a new programming language [Hoare 2022]. The language, soon to be known as Rust, had two goals. First, to be system-oriented, meaning the programmer would deal with references, pointers, and manually manage memory. Second, to be safe, meaning the compiler's static discipline would rule out memory errors such as use-after-free,

Authors' addresses: Son Ho, Inria, France, son.ho@inria.fr; Jonathan Protzenko, Microsoft Research, USA, protz@microsoft.com.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2022 Copyright held by the owner/author(s).

2475-1421/2022/8-ART116

<https://doi.org/10.1145/3547647>

or arbitrary memory access. Even though the language evolved a great deal since its inception, these two core premises remain today.

Sixteen years later, Rust enjoys a substantial amount of success, and has ranked as the most loved programming language for six consecutive years on StackOverflow’s developer survey [sta 2021]. But as the systems community can attest [Bhargavan et al. 2017; Ferraiuolo et al. 2017; Klein et al. 2009; Lorch et al. 2020], memory safety is too weak of a property, no matter how remarkable of an achievement Rust is. Indeed, we oftentimes want to prove deep *correctness properties* of a system. Doing so may involve anything from baseline safety properties, such as the absence of assertion failures or runtime errors, to complex invariants involving concurrent systems.

As a consequence, several verification toolchains have emerged to facilitate proving deep properties about low-level programs. For pragmatic reasons, C is oftentimes a target of choice [Cao et al. 2018; Protzenko et al. 2017]; so is using a custom language, such as Dafny [Hawblitzel et al. 2014]. Alas, whether the tool is based on separation logic [Reynolds 2002] or modifies-clauses [Leino 2010], verification engineers soon find themselves drowning under a sea of memory-related obligations that distract them from the properties of interest. The net result is that verification engineers spend an undue amount of time discharging mundane memory-related proof obligations, because the frontend language does not enforce enough invariants to begin with.

One strategy is to write a checker for an existing language, restricting its usage enough that verification becomes easier. This is the strategy used by e.g., Linear Dafny [Li et al. 2022] or RefinedC [Sammler et al. 2021]. Another strategy is to leverage invariants provided for free by languages with restrictive type systems; that is, to leverage a language like Rust.

Rust is, in effect, trying to reconcile systems programming, and a long tradition of static ownership disciplines [Boyland et al. 2001; Clarke et al. 1998] which traces back to linear types [Wadler 1990], regions [Tofte and Talpin 1997] and the combination thereof [Fluet et al. 2006]. Today, Rust incorporates *ownership* in all aspects of its design; notably, programmers rely on *borrowing* to take references with ownership, and the type system relies on a notion of *lifetime* to enforce the soundness of borrows. This static discipline aims to give the programmer maximum flexibility for common idioms, while still preventing arbitrary aliasing.

The adequacy of Rust as a verification target has not gone un-noticed. There are now several research projects aiming to set up verification frameworks for Rust, using a variety of backends, such as SMT [ver 2022; Matsushita et al. 2020], Viper [Astrauskas et al. 2019; Wolff et al. 2021], or Why [Denis et al. 2021]. With these works, we have gained a deeper understanding of why verifying Rust programs remains difficult, even in the presence of Rust’s strong ownership discipline.

First, Rust’s type system is not *simply* linear, and features a rich variety of mechanisms, such as: reborrows, two-phase borrows, functions returning borrows, along with many other subtle rules that ensure most common idioms go through the type-checker smoothly. Accounting for all of these is notoriously difficult. Second, the borrow mechanism introduces non-locality, in that ending a borrow requires propagating knowledge backwards, in order to update the previously-borrowed variable with a new value. This central difficulty is handled via a variety of technical devices in other works, such as prophecy variables [Matsushita et al. 2020] or *after* clauses [ver 2022].

In this paper, we propose a new approach to understanding and verifying Rust programs. At the heart of our methodology is a lightweight functional translation of Rust programs. We eschew the complexity of connecting to a separation-logic based backend [Jung et al. 2017], or relying on prophecy variables to produce a logical encoding [Matsushita et al. 2022, 2020]. Instead, we synthesize a pure, functional, executable equivalent of the original Rust program, thus producing a lambda-term that does not rely on memory or special constructs. Our translation handles shared, mutable, two-phase and re-borrows, and thus accounts for a very large fraction of typical Rust programs. We call the conceptual framework, as well as the companion tool, AENEAS.

We wish to emphasize that our functional translation is completely generic. While we demonstrate a possible verification backend by printing our pure programs in F^* syntax, many other options are possible. One could easily add additional backends (Coq, Lean, Viper, Why) or devise a contract language for source Rust programs that directly emits SMT proof obligations using our translation. We certainly hope to write some of these in the near future.

To elaborate on the design choices made by AENEAS: we intentionally focus on programs that abide by Rust’s static ownership discipline. That is, we do not tackle `unsafe` blocks – we believe such programs are better suited to a sophisticated framework such as RustBelt. We do not tackle interior mutability either, in which the user can use unfettered aliasing, in exchange for a run-time borrow checker. We wish to focus instead on the Rust subset that is functional in essence, meaning we leave treatment of interior mutability up to future work. We believe this places us in a “sweet spot” for verifying Rust programs. The key observation of this work is that for the most part, references and borrows serve the purpose of optimizing either performance (e.g., passing by reference instead of by value), or memory representation (e.g., by controlling aliasing and taking inner pointers within data structures). That is, Rust’s references do not serve any semantic purpose; coupled with the fact that the type system is enforcing a linear discipline, such programs are functional in essence, and can be naturally translated to a pure functional equivalent. Wadler observed that a linear type system allows compiling pure programs using imperative updates [Wadler 1990]; we leverage the reverse observation [Charguéraud and Pottier 2008], that is, imperative programs with a strong enough ownership discipline admit a functional equivalent.

To reiterate: the key point of this work is that we give a *functional* semantics, and thus a *pure* translation, to the subset of Rust we consider. Concretely, we define LLBC, the Low-Level Borrow Calculus, to model that subset. Then, we give it an operational semantics that is functional in nature. We do not rely on memory, addresses or pointer arithmetic; rather, we map variables to values, and track aliasing in a very fine-grained manner. We claim that our operational semantics captures the *essence* of borrowing; that is, it does not simply apply the rules dictated by Rust’s lifetime discipline. Rather, it establishes what is allowed with regards to ownership in the presence of moves, borrows and copies. As such, our semantics can account not only for the current borrow-checker’s behavior, but also for its future evolutions, such as Polonius [The Rust Compiler Team 2021].

Our functional semantics paves the way for our functional translation. We proceed in two steps. First, we tweak our semantics to abstract the aliasing graph in the presence of function calls; to do so, we interpret regions as bags of borrows and loans. Next, we follow the structure of the program in the presence of these region abstractions, and generate a functional translation. To overcome the key difficulty of terminating a borrow, we rely on a technical innovation called *backward functions*, which obviates the need for prophecy variables (as in RustHorn), or *after* clauses (as in Verus).

We have implemented our functional translation approach in a mixture of Rust and OCaml, for a total of 10,000 and 14,000 lines of code, respectively (excluding comments and whitespace). Once the pure translation is synthesized, we emit pure code for the theorem prover of our choice: we currently support F^* , and we have a Coq backend in the works.

We evaluate AENEAS on a wide variety of micro-benchmarks for feature-completeness, and verify a resizable hash table as our main case study. We find that the benefits of AENEAS are many. First, with AENEAS, the verification engineer deals with the *intrinsic* difficulty of the proofs, rather than the *incidental* complexity; that is, they can focus on the essence of the proof rather than being mired in the technicalities of memory reasoning. There are no modifies-clause lemmas, and no separation logic framing tactics; these are, by construction, un-necessary. Second, our approach remains *lightweight*: we do not need to design an annotation language for Rust programs, and the properties we prove are not constrained by the expressivity or the usability of said annotation language. Consequently, AENEAS-translated programs can easily integrate within an existing project, and

leverage libraries or proof tactics, as opposed to evolving in a closed world whose boundaries are set by a specific annotation language. Third, we are not beholden to one specific verification framework; we envision a world where the verification engineer can simply direct AENEAS towards the proof framework they are the most productive in.

In short, AENEAS reveals the functional essence of Rust programs, and verifies them as such.

We start with an accessible, example-based introduction to Rust and AENEAS (Section 2), then present this paper’s contributions:

- an ownership-centric operational semantics for Rust (Section 3),
- a concept of *region abstraction*, which precisely models the interaction of borrows, regions and function calls; region abstractions establish a blueprint for our functional translation, which relies on what we dub *backward functions* (Section 4),
- a complete implementation, based on a Rust plugin and a subsequent OCaml compiler named AENEAS (Section 5); the former extracts all the information we need from the internal Rust AST, while the latter implements all of the steps described in this paper,
- an experimental evaluation that culminates in the verification of a resizable hash table; to the best of our knowledge, the man-hours spent proving that the Rust hash table functionally behaves like a map are extremely modest relative to other similar efforts: AENEAS thus offers substantial gains of productivity (Section 6).

We acknowledge the current limitations of our tool; situate AENEAS relative to other Rust verification efforts, and conclude (Section 7). The implementation of our tools, the verified code and a long version of this paper are available online [Ho and Protzenko 2022a,b,c].

2 AENEAS AND ITS FUNCTIONAL TRANSLATION, BY EXAMPLE

Before jumping into the various facets of our formalism, we keep an eye on the prize, and immediately showcase how AENEAS translates Rust programs to pure equivalents. In this section, and for the remainder of the paper, we use F* syntax for our functional translation; it greatly resembles OCaml and other ML languages, and as such should be familiar to the reader. A brief note about terminology: we adopt the view of Matsakis [2018], and refer to *regions*, emphasizing that a region encompasses a set of borrows and loans at a given program point. The Rust compiler and documentation, however, refer to *lifetimes*, which conveys the idea of a syntactic bracket, and a specific implementation technique to enforce soundness. In this paper, whenever we talk about Rust specifically, we use “lifetime”; whenever we emphasize our semantic view of ownership, we use “region”.

Mutable Borrows, Functionally. To warm up, we consider an example that, albeit small, showcases many of Rust’s features, including its ownership mechanism. In the Rust program below, `ref_incr` increments a reference, and `test_incr` acts as a representative caller of the function.

```

1 fn ref_incr(x: &mut i32) {
2     *x = *x + 1; }
3
4 fn test_incr() {
5     let mut y = 0i32;
6     ref_incr(&mut y);
7     assert!(y == 1); }
```

The `incr` function operates by reference; that is, it receives the address of a 32-bit signed integer `x`, as indicated by the `&` (reference) type. In addition, `incr` is allowed to modify the contents at address `x`, because the reference is of the `mut` (mutable) kind, which permits memory modification. Finally, the Rust type system enforces that mutable references have a unique owner: the definition of `ref_incr`

type-checks, meaning that the function not only *guarantees* it does not duplicate ownership of x , but also can *rely* on the fact that no one else owns x .

In `test_incr`, we allocate a mutable value (`let mut`) on the stack; upon calling `ref_incr`, we take a mutable reference (`&mut`) to y . Statically, y becomes unavailable as long as `&mut y` is active. In Rust parlance, y is *mutably borrowed* and its ownership has been transferred to the mutable reference. To type-check the call, the type-checker performs a lifetime analysis: the `ref_incr` function has type `(&'a mut i32) -> ()`, and the `&mut y` borrow has type `&'b mut i32`; both `'a` and `'b` are lifetime variables.

For now, suffices to say that the type-checker ascertains that the lifetime `'b` of the mutable borrow satisfies the lifetime annotation `'a` in the type of the callee, and deems the call valid. Immediately after the call, Rust *terminates* the region `'b`, in effect *relinquishing* ownership of the mutable reference `&'b mut y` so as to make y usable again inside `test_incr`. This in turn allows the `assert` to type-check, and thus the whole program. Undoubtedly, this is a very minimalistic program; yet, there are two properties of interest that we may want to establish already. The obvious one: the assertion always succeeds. More subtly, doing so requires us to prove an additional property, namely that the addition at line 2 does not overflow.

The key insight of AENEAS is that even though the program manipulates references and borrows, none of this is informative when it comes to reasoning about the program. More precisely: x and y are uniquely owned, meaning that there are no stray aliases through which x or y may be modified; in other words, to understand what happens to y , it suffices to track what happens to `&mut y`, and therefore to x . Feeding this program to AENEAS generates the following translation, where `i32_add` is an AENEAS primitive that captures the semantics of error-on-overflow in Rust.

```

1 let ref_incr_fwd (x : i32): result i32 =
2   match i32_add x 1 with | Fail -> Fail | Return x0 -> Return x0
3
4 let test_incr : result unit =
5   match ref_incr_fwd 0 with
6   | Fail -> Fail
7   | Return y -> if not (y = 1) then Fail else Return ()
8
9 let _ = assert (test_incr = Return ())

```

This program is semantically equivalent to the original Rust code, but does not rely on the memory: we have leveraged the precise ownership discipline of Rust to generate a functional, pure version of the program. In hindsight, the usage of references in Rust was merely an implementation detail, which is why `ref_incr_fwd` becomes a simple (possibly-overflowing) addition. Should the call to `ref_incr_fwd` (line 5) succeed, its result is bound to y (line 7); the `assert` simply becomes a boolean test that may generate a failure in the error monad.

For the purposes of unit-testing, AENEAS inserts an additional assertion for `test_*` functions of type `unit -> unit`: the prover shows instantly that our test always succeeds. (In F^* , we execute this assertion directly on the normalizer, without even resorting to SMT; AENEAS produces an executable translation, not a logical encoding.) In the remainder of this section, we use $\leftarrow$, F^* 's bind operator¹ in the error monad.

Returning a Mutable Borrow, and a Backward Function. Rust programs, however, rarely admit such immediate translations. To see why, consider the following example, where the `choose` function returns a borrow, as indicated by its return type `&'a mut`.

```

1 fn choose<'a, T>(b: bool, x: &'a mut T, y: &'a mut T) -> &'a mut T {
2   if b { return x; } else { return y; } }
3
4 fn test_choose() {

```

¹There are issues in F^* related to this notation [fst 2017], which we work around in practice, but ignore here.


```

5   let mut x = 0i32; let mut y = 0i32;
6   let z = choose(true, &mut x, &mut y);
7   *z = *z + 1;
8   assert!(*z == 1);
9   assert!(x == 1); assert!(y == 0); }

```

The `choose` function is polymorphic over type τ and lifetime $'a$; the lifetime annotation captures the expectation that both x and y be in the same region. At call site, x and y are borrowed (line 6): they become unusable, and give birth to two intermediary values `&mut x` and `&mut y` of type $\&'a \text{ mut } i32$. The value returned by `choose` also lives in region $'a$, i.e., z also has type $\&'a \text{ mut } i32$. The usage of z (lines 7-8) is valid because the region $'a$ still exists; the Rust type-checker infers that region $'a$ ought to be terminated after line 8, which ends the borrows and therefore allows the caller to regain full ownership of x and y , so that the `asserts` at line 9 are well-formed.

At first glance, it appears we can translate `choose` to an obvious conditional. But if we reason about the semantics of `choose` from the caller's perspective, it turns out that the intuitive translation is not sufficient to capture what happens, e.g., to x and y at lines 9. At call site, `choose` is an opaque, separate function, meaning the caller cannot reason about its precise definition – all that is available is the function type. This type, however, contains precise region information. When performing the function call, the ownership of x and y is transferred to region $'a$ in exchange for z ; symmetrically, when the lifetime $'a$ terminates, z is relinquished to region $'a$ in exchange for regaining ownership of x and y . The former operation flows *forward*; the latter flows *backward*. Using a separation-logic oriented analogy: borrows and regions encode a magic wand that is introduced in a function call and eliminated when the corresponding region terminates.

Our point is: both function call and region termination are semantically meaningful. In our earlier example, the `ref_incr` function returned a unit, meaning that AENEAS only emitted a *forward* function (hence the `_fwd` suffix) to translate the function call. With the `choose` example, AENEAS emits both a *forward* and a *backward* function, used for the function call and the end of the $'a$ region, respectively.

```

1  let choose_fwd (t : Type) (b : bool) (x : t) (y : t) : result t =
2    if b then Return x else Return y
3
4  let choose_back (t : Type) (b : bool) (x : t) (y : t) (ret : t) : result (t & t) =
5    if b then Return (ret, y) else Return (x, ret)
6
7  let test_choose_fwd : result unit =
8    i <-- choose_fwd i32 true 0 0;
9    z <-- i32_add i 1;
10   massert (z == 1); (* monadic assert *)
11   (x0, y0) <-- choose_back i32 true 0 0 z;
12   massert (x0 == 1);
13   massert (y0 == 0);
14   Return ()
15
16 let _ = assert (test_choose_fwd == Return ())

```

The call to `choose` becomes a call to the forward function `choose_fwd` (line 8); we bind the result of the addition (provided no overflow occurs) to z (line 9); then, per the rules of Rust's type-checker, region $'a$ terminates which compels us to call the backward function `choose_back`. The intuitive effect of calling `choose_back` is as follows: we relinquish z , which was in region $'a$; doing so, we propagate any updates that may have been performed through z onto the variables whose ownership was transferred to $'a$ in the first place, namely x and y . This bidirectional approach is akin to lenses [Bohannon et al. 2008], except we propagate the output back to possibly-many

inputs; in this case, z is a view over either x or y , and the backward function reflects the update to z onto the original variables. Thus, both variables are re-bound (line 11), before chaining the two asserts (lines 12 and 13).

From the caller's perspective, the computational content of `choose` is unknown; but the signature of `choose` reveals the effect it may have onto its inputs x and y , which in turns allows us to derive the type of the backward and forward functions from the signature of `choose` itself. The result is a modular, functional translation that does not rely on any sort of cross-function inlining or whole-program analysis. To synthesize `choose_back`, it suffices to invert the direction of assignments; in one case, z flows to x and y remains unchanged; the other case is symmetrical.

Recursion and Data Structures. It might not be immediately obvious that this translation technique scales up beyond toy examples; to conclude this section, we crank up the complexity and show how AENEAS can handle a wide variety of idioms while still delivering on the original promise of a lightweight functional translation. Our final example is `list_nth`, which allows taking a mutable reference to the n -th element of a list, mutating it, and regaining ownership of the list.

```

1 enum List<T> { Cons(T, Box<List<T>>), Nil }
2
3 fn list_nth_mut<'a, T>(l: &'a mut List<T>, i: u32) -> &'a mut T {
4     match l {
5         Nil => { panic!() }
6         Cons(x, tl) => { if i == 0 { x } else { list_nth_mut(tl, i - 1) } } }
7
8 fn sum(l: & List<i32>) -> i32 {
9     match l {
10        Nil => { return 0; }
11        Cons(x, tl) => { return *x + sum(tl); } }
12
13 fn test_nth() {
14     let mut l = Cons(1, Box::new(Cons(2, Box::new(Cons(3, Box::new(Nil))))));
15     let x = list_nth_mut(&mut l, 2);
16     *x = *x + 1;
17     assert!(sum(&l) == 7); }

```

This example relies on several new concepts. Parametric data type declarations (line 1) resemble those in any functional programming language such as OCaml or SML. The `Box` type denotes a heap-allocated, uniquely-owned piece of data. Without the `Box` indirection, `List` would describe a type of infinite size and would be rejected. Immutable borrows (line 8) do not sport a `mut` keyword; they do not permit mutation, but the programmer may create infinitely many of them. Only when all shared borrows have been relinquished does full ownership return to the borrowed value. The complete translation is in our long version [Ho and Protzenko 2022c]; the salient part comes out of AENEAS as follows:

```

1 type list_t (t : Type) = | ListCons : t -> list_t t -> list_t t | ListNil : list_t t
2
3 let rec list_nth_mut_back (t : Type) (l : list_t t) (i : u32) (ret : t) : result (list_t t) =
4     match l with
5     | ListNil -> Fail
6     | ListCons x tl -> match i with
7         | 0 -> Return (ListCons ret tl)
8         | _ -> i0 <- u32_sub i 1;
9             l0 <- list_nth_mut_back t tl i0 ret;
10             Return (ListCons x l0)
11
12 let test_nth_fwd : result unit =
13     let l2 = ListCons (1, ListCons (2, ListConst (3, ListNil))) in

```



```

14  i <-- list_nth_mut_fwd i32 12 2;
15  x <-- i32_add i 1;
16  l2 <-- list_nth_mut_back i32 12 2 x;
17  i0 <-- sum_fwd l2;
18  massert (i0 = 7)

```

We first focus on the caller's point of view. Continuing with the lens analogy, we focus on (or “get”) the n -th element of the list via a call to `list_nth_mut_fwd` (line 14); modify the element (line 15); then close (or “put” back) the lens, and propagate the modification back to the list via a call to `list_nth_mut_back` (line 16). The `list_nth_mut_back` function is of particular interest. The function follows the same control-flow as the forward function. However, the key part happens when returning from the `Cons` case: our functional translation performs a *semantic* region analysis, from which it follows that a recursive call to the backward function is needed, along with the construction of a new `Cons` cell.

3 AN OWNERSHIP-CENTRIC SEMANTICS FOR RUST

Before explaining the functional translation above, we must first define our input language and its operational semantics. We now present a series of short Rust snippets, and show in comments how our execution environments model the effect of each statement.

Mutable borrows. After line 1, x points to 0, which we write $x \mapsto 0$. At line 2, px *mutably* borrows x . As we mentioned earlier, a mutable borrow grants exclusive ownership of a value, and renders the borrowed value unusable for the duration of the borrow. We reflect this fact in our execution environment as follows: x is marked as “loaned-out”, in a mutable fashion, and px is known to be a mutable borrow. Furthermore, ownership of the borrowed value now rests with px , so the value within the mutable borrow is 0. Finally, we need to record that px is a borrow *of* x : we issue a fresh loan identifier ℓ that ties x and px together. The same operation is repeated at line 3. Value 0 is now held by ppx , and px , too, becomes “loaned out”.

```

1  let mut x = 0;           // x ↦ 0
2  let mut px = &mut x;    // x ↦ loanm ℓ,  px ↦ borrowm ℓ 0
3  let ppx = &mut px;      // x ↦ loanm ℓ,  px ↦ loanm ℓ',  ppx ↦ borrowm ℓ' (borrowm ℓ 0)

```

Our environments thus precisely track ownership; doing so, they capture the aliasing graph in an exact fashion. Another point about our style: this representation allows us to adopt a *focused* view of the borrowed value (e.g. 0), solely through its owner (e.g. ppx), without worrying about following indirections to other variables. We believe this approach is unique to our semantics; it has, in our experience, greatly simplified our reasoning and in particular the functional translation (Section 4).

We remark that our style departs from Stacked Borrows [Jung et al. 2019], where the modified value remains with x . We also note that our formalism cannot account for unsafe blocks; allowing unfettered aliasing would lead to potential cycles, which we cannot represent. This is an intentional design choice for us: we circumscribe the problem space in order to achieve an intuitive, natural semantics and a lightweight functional translation. AENEAS shines on non-unsafe Rust programs, and can be complemented by more advanced tools such as RustBelt for unsafe parts.

Shared borrows. Shared borrows behave more like traditional pointers. Multiple shared borrows may be created for the same value; each of them grants read-only access to the underlying value. Regaining full ownership of the borrowed value requires terminating all of the borrows. In the example below, the value (0, 1) is borrowed in a shared fashion, at line 2. This time, the value remains with x ; but taking an immutable reference to x still requires book-keeping. We issue a new loan ℓ , and record that $px1$ is now a shared borrow associated to loan ℓ ; to understand which value $px1$ points to, we simply look up in the environment who is the owner of ℓ , and read the associated value. Repeated shared borrows are permitted: at line 3, we issue a new loan ℓ' which

augments the loan-set of x . At line 4, we anticipate on our internal syntax, where moves and copies are explicit, and copy the first component of x . Values that are loaned immutably, like x , can still be read; in the resulting environment, y points to a copy of the first component, and bears no relationship whatsoever to x . Finally, at line 5, we *reborrow* (through $px1$) the first component of the pair only. First, to dereference $px1$, we perform a lookup and find that x owns ℓ . Then, we perform book-keeping and update the value loaned by x , so as to reflect that its first component has been loaned out.

```

1 let x = (0, 1);      //  $x \mapsto (0, 1)$ 
2 let px1 = &x;        //  $x \mapsto \text{loan}^s \{ \ell \} (0, 1), \quad px1 \mapsto \text{borrow}^s \ell$ 
3 let px2 = &x;        //  $x \mapsto \text{loan}^s \{ \ell \cup \ell' \} (0, 1), \quad px1 \mapsto \text{borrow}^s \ell, \quad px2 \mapsto \text{borrow}^s \ell'$ 
4 let y = copy x.0;    //  $x \mapsto \text{loan}^s \{ \ell \cup \ell' \} (0, 1), \quad px1 \mapsto \text{borrow}^s \ell, \quad px2 \mapsto \text{borrow}^s \ell', \quad y \mapsto 0$ 
5 let z = &(px1.0);    //  $x \mapsto \text{loan}^s \{ \ell \cup \ell' \} ((\text{loan}^s \{ \ell'' \} 0), 1), \quad px1 \mapsto \dots, \quad px2 \mapsto \dots, \quad y \mapsto 0, \quad z \mapsto \text{borrow}^s \ell''$ 

```

Finally, we reiterate our remark that our formalism allows keeping track of the aliasing graph in a precise fashion; the discipline of Rust bans cycles, meaning that the aliasing graph is always a tree. This style of representation resembles Mezzo [Balabonski et al. 2016], where loan identifiers are akin to singleton types, and entries in the environment are akin to permissions.

Rewriting an Old Value, a.k.a. Reborrowing. We now consider a particularly twisted example accepted by the Rust compiler. While the complexity seems at first gratuitous, it turns out that the pattern of borrowing a dereference (i.e., `&mut (*px)`) is particularly common in Rust. The reason is subtle: in the post-desugaring MIR internal Rust representation, moves and copies are explicit, meaning function calls of the form `f(move px)` abound. Such function calls *consume* their argument, and render the user-declared reference `px` unusable past the function call. To offer a better user experience, Rust automatically “reborrow” the contents pointed to by `px`, and rewrites the call into `f(move (&mut (*px)))` at desugaring-time. Thus, only the intermediary value is “lost” to the function call; relying on its lifetime analysis, the Rust compiler concludes that the user-declared reference `px` remains valid past the function call, hence making the programmer’s life easier.

Another common pattern is to directly mutate a borrow, i.e. assign a fresh borrow into a variable x of type `&mut t` that was *itself* declared as `let mut`. Capturing the semantics of such an update must be done with great care, in order to preserve precise aliasing information.

We propose an example that combines both patterns; the fact that we make `px` reborrow itself is what makes the example “twisted”. Rust accepts this program; we now explain with our semantics *why* it is sound. In the example below, after line 2, the environment offers no surprises. Justifying the write at line 3 requires care. We borrow `*px`, which modifies `px` to point to `borrowm  $\ell'$  0`, and returns `loanm  $\ell'$` ; the value about to be overwritten is stored in a fresh variable `pxold`, and `loanm  $\ell'$` gets written to `px`.

```

1 let mut x = 0;      //  $x \mapsto 0$ 
2 let mut px = &mut x; //  $x \mapsto \text{loan}^m \ell, \quad px \mapsto \text{borrow}^m \ell 0$ 
3 px = &mut (*px);    //  $x \mapsto \text{loan}^m \ell, \quad px_{old} \mapsto \text{borrow}^m \ell (\text{loan}^m \ell'), \quad px \mapsto \text{borrow}^m \ell' 0$ 
4                          //  $x \mapsto \text{loan}^m \ell, \quad px_{old} \mapsto \text{borrow}^m \ell 0, \quad px \mapsto \perp$ 
5 assert!(x==0);     //  $x \mapsto 0, \quad px_{old} \mapsto \perp, \quad px \mapsto \perp$ 

```

Saving the old value is crucial for line 4. For the assertion, we need to regain full ownership of x . To do so, we first terminate ℓ' . This *reorganizes* the environment, with two consequences. First, `px` becomes unusable, which we write `px  $\mapsto \perp$` . Second, `pxold`, which we had judiciously kept in the environment, becomes `borrowm  $\ell 0$` . We reorganize the environment again, to terminate ℓ ; the effect is similar, and results in `x  $\mapsto 0$` , i.e. full ownership of x . This example illustrates a key characteristic of our approach, which is that we reorganize borrows in a lazy fashion, and don’t terminate a borrow until we need to get the borrowed value back.

Syntax			
$\tau ::=$	type	$op ::=$	operand
bool uint32 int32	base types	move p	ownership transfer
$\&^p \text{ mut } \tau$	mutable borrow	copy p	scalar copy
$\&^p \tau$	immutable borrow	true false n_{i32} n_{u32}	constants
$T \vec{\tau}$	type application	$C[\vec{f} = \vec{op}]$	data type constructor
$\alpha, \beta, \dots$	type variables	$(\vec{op})$	tuple or unit
$(\vec{\tau})$	tuple or unit	x	variable
$T ::=$	type application	$p ::= P[x]$	place
t	user-defined data type	$P ::=$	path
Box	boxed type	$[\cdot]$	base case
$s ::=$	statement	$*^s p$	deref shared borrow
$\emptyset$	empty statement (nil)	$*^m p$	deref mutable borrow
$s; s$	sequence (cons)	$*^b p$	deref box
$p := rv$	assignment	$P.f$	field selection
$p := f \vec{op}$	function call	$P.n$	field selection (tuple)
if op then s else s	conditional	$D ::=$	top-level declaration
match p with $\overrightarrow{C \rightarrow s}$	case analysis	fn $f \langle \vec{\rho} \rangle$	
return	function exit	$(\vec{x}_{\text{arg}} : \vec{\tau})$	
panic	unrecoverable error	$(\vec{x}_{\text{local}} : \vec{\tau})$	function declaration
$\dots$	loops, etc.	$(x_{\text{ret}} : \vec{\tau})$	
$rv ::=$	assignable “r” values	$= s$	
op	operand	type $t \vec{\alpha} = C[\vec{f} : \vec{\tau}] \mid \dots$	data type declaration
$\&\text{mut } p$	mutable borrow		
$\&p$	immutable borrow		
$!op \mid op + op \mid op - op$	operators		
Reduction			
$v ::=$	value	ℓ	loan identifier
true false n_{i32} n_{u32}	booleans and integers	$\Omega ::=$	evaluation context
borrow $^m \ell \ v$	mutable borrow	$\emptyset$	empty
borrow $^s \ell$	shared borrow	$x \mapsto v, \Omega$	new mapping
borrow $^r \ell$	reserved borrow		
loan $^m \ell$	mutable loan		
loan $^s \{\vec{\ell}\} \ v$	shared loan		
$\perp$	inaccessible value		
$C[\vec{f} = \vec{v}]$	data type constructor		
$(\vec{v})$	tuple constructor or unit		
$r ::=$	result		
panic	unrecoverable error state		
return v	successful, possibly-early exit		
$\dots$	loop states, etc.		

Fig. 1. The Low-Level Borrow Calculus: Syntax, Reduction Environments, Values

3.1 The Low-Level Borrow Calculus

We now formally introduce and define our semantics of Rust programs. We start with the Low-Level Borrow Calculus (“LLBC”, Figure 1), our source language. LLBC is in large part inspired by MIR, Rust’s post-desugaring internal representation, notably: all local variables $\vec{x}_{\text{local}}$ are bound at

$$\begin{array}{c}
\text{READ} \\
\frac{p = P[x] \quad x \mapsto v_x \in \Omega \quad \Omega \vdash P(v_x) \Rightarrow v}{\Omega(p) \Rightarrow v} \\
\\
\text{R-MUT-BORROW} \\
\frac{\Omega \vdash P(vp) \Rightarrow v}{\Omega \vdash (*^m P)(\text{borrow}^m \ell \ v_p) \Rightarrow v} \\
\\
\text{R-FIELD} \\
\frac{\Omega \vdash P(vp) \Rightarrow v}{\Omega \vdash (P.f)(C[f = v_p]) \Rightarrow v} \\
\\
\text{WRITE} \\
\frac{x \mapsto v_x \in \Omega \quad p = P[x] \quad \Omega \vdash P(v_x) \leftarrow v \Rightarrow v'_x \quad \Omega' = \Omega[x \mapsto v'_x]}{\Omega(p) \leftarrow v \Rightarrow \Omega'} \\
\\
\text{W-MUT-BORROW} \\
\frac{\Omega \vdash P(vp) \leftarrow v \Rightarrow v'_p}{\Omega \vdash (*^m P)(\text{borrow}^m \ell \ v_p) \leftarrow v \Rightarrow \text{borrow}^m \ell \ v'_p} \\
\\
\text{R-BOX} \\
\frac{\Omega \vdash P(vp) \Rightarrow v}{\Omega \vdash (*^b P)(\text{Box } vp) \Rightarrow v} \\
\\
\text{R-SHARED-BORROW} \\
\frac{\text{loan}^s \{\ell \cup _ \} \ v_p \in \Omega \quad \Omega \vdash P(vp) \Rightarrow v}{\Omega \vdash (*^s P)(\text{borrow}^s \ell) \Rightarrow v} \\
\\
\text{R-SHARED-LOAN} \\
\frac{P \neq [_] \quad \Omega \vdash P(vp) \Rightarrow v}{\Omega \vdash P(\text{loan}^s \{ _ \} \ v_p) \Rightarrow v} \\
\\
\text{W-BOX} \\
\frac{\Omega \vdash P(vp) \leftarrow v \Rightarrow v'_p}{\Omega \vdash (*^b P)(\text{Box } vp) \leftarrow v \Rightarrow \text{Box } v'_p} \\
\\
\text{W-BASE} \\
\frac{}{\Omega \vdash [_](vp) \leftarrow v \Rightarrow v}
\end{array}$$

Fig. 2. Reading From and Writing To Our Structured Memory Model

the beginning of the function declaration; returning a value from a function amounts to writing into the special variable x_{ret} , followed by return; all subexpressions have been named so as to fit within MIR's statement/r-value/operand categories; and all variables within expressions have been desugared to either a move, a copy or a borrow. However, and unlike MIR, LLBC retains some high-level constructs: control-flow remains structured (LLBC statements are thus the fusion of MIR's statements and terminators); and case analysis on data types is exposed via a limited form of (complete) pattern matching, as opposed to a low-level integer switch on the tag. We remark that data types may match on a *path* only; this merely imposes that the scrutinee be let-bound before examining it, something that MIR does internally. Finally, we use pure expressions to allocate data types, rather than the progressive (mutable) initialization pattern used by MIR; and we see structures as data types equipped with a single constructor, for conciseness. Staying close to MIR is a design choice in line with other Rust-related works [Jung et al. 2019]; it allows for fewer, simpler rules, and a more precise description of what happens from the point of view of ownership.

At the heart of LLBC is a notion of *place*, i.e. the combination of a base variable (e.g. x) and a series of field offsets and indirections known as a *path* (e.g. $*_.f$). A place is akin to the notion of "lvalue" in, e.g., C. Assigning or returning from a function can only be done into a *place*. The grammar of rvalues and operands (Rust's limited form of expression) is very explicit, in that every use of a variable is performed through a copy, a move, or a borrow of a place.

3.2 A Structured Memory Model

Rust marries high-level concepts, such as ownership, a strong notion of value, and data types, with low-level concepts such as moves and copies, paths through a base address, and modifications at depth throughout the store. We propose a semantics that operates exclusively in terms of values (that is, no memory addresses), yet still permits fine-grained memory mutations as allowed by Rust.

Figure 1 presents our environments, which we write Ω , and our values, which we write v . We do not distinguish between store and environment, and use the two terms interchangeably. The store maps variable names x to values v : we have no notion of arbitrary memory addresses, or pointer arithmetic. Our values v are carefully crafted to model the semantics of borrows and ownership tracking in Rust; several of them already appeared in our earlier examples.

The combination of places, environments and values allows us to define reads and writes already. Reads and writes are defined in terms of our *structured* memory model: we do not have any notion of memory address, but *do* have a notion of path combined with a base “address” (variable) x , that is, a place; this permits reads and writes, at depth, through references. We present selected rules for reading (R-*) and writing (W-*) in Figure 2.

For *reading*, we write $\Omega(p) \Rightarrow v$, meaning reading from Ω at place p produces v . Doing so requires looking up the “base pointer” (variable) x found in p (READ), then deferring to an auxiliary judgment of the form $\Omega \vdash P(v_x) \Rightarrow v$, meaning following path P into v_x produces value v . We can follow path P as long as the value v_x is of the right shape (R-MUT-BORROW, R-FIELD). Reading from a mutable borrow requires no additional operation, since the mutable borrow uniquely owns the value it points to (R-MUT-BORROW). Conversely, reading from a shared borrow requires looking up the owner of the loan to find the value being pointed to (R-SHARED-BORROW).

Rule R-BASE is our base case: if we have reached the end of the path P , we simply return the value found there. One subtlety occurs in the case of shared *loans*. Rule R-SHARED-LOAN permits reading from a value that is currently immutably borrowed, which is allowed in Rust. However, we only do so if necessary; that is, if we must follow further indirections in P (i.e., $P \neq [.]$). If there are no further indirections (i.e., $P = [.]$), R-BASE kicks in and returns a value of the form loan^s . This is intentional, and will prove useful for E-SHARED-OR-RESERVED-BORROW, as we shall see shortly.

For *writing*, we write $\Omega(p) \leftarrow v \Rightarrow \Omega'$, meaning assigning value v into Ω at place p produces an updated environment Ω' (WRITE). As before, we follow the structure of v_x (WRITE), and defer to an auxiliary judgment of the form $\Omega \vdash P(v_x) \leftarrow v \Rightarrow v'_x$, which from v_x computes an updated value v'_x where only the subexpression selected by P is updated with v . We update x 's entry in the environment to map to the new value v'_x , denoted $\Omega[x \mapsto v'_x]$ (WRITE). As before, the shape of P determines which rule applies: we may only write through a mutable borrow (W-MUT-BORROW) or a box (W-BOX). We eventually apply W-BASE. We elide the remaining rules (tuples, fields, etc.).

3.3 Semantics of Ownership and Borrows

At the heart of our operational semantics is our treatment of borrows, which captures ownership transfer. We now introduce the core of our operational rules in Figure 3, and describe in detail the essential operations: borrows, moves, copies, and assignments. Our rules start with E-, for evaluation rules. Both the rules and our earlier syntax manipulate a third flavor of borrows, which we dub “reserved” borrows, denoted as borrow^r . A technical device, they account for the two-phase borrows introduced by the Rust compiler in the process of desugaring to MIR. We explain those later, in Section 3.4; they can be safely ignored for now.

A few preliminary remarks about notation: we rely on the auxiliary notion of a *ghost update*, denoted $\Omega[p \mapsto v]$ – this extends our earlier notation of $\Omega[x \mapsto v]$ – for updating an entry in the environment. In contrast to run-time writes, previously introduced as $\Omega(p) \leftarrow v \Rightarrow \Omega'$ (WRITE), ghost updates do not have any effect at run-time (Figure 5). Instead, they allow us to perform the necessary book-keeping to statically keep track of ownership and aliases. The definition of ghost updates is almost identical to run-time writes (WRITE), the only difference being that ghost updates can follow a shared borrow to perform an administrative update underneath the shared loan. As we will see shortly, this is leveraged by e.g. E-SHARED-OR-RESERVED-BORROW, to track borrowing underneath a shared borrow. We write $\Omega \vdash op \rightsquigarrow v \dashv \Omega'$ to indicate that in environment Ω , operand

$$\begin{array}{c}
\text{E-MUT-BORROW} \\
\frac{\Omega(p) \Rightarrow v \quad \{\perp, \text{loan}, \text{borrow}^r\} \notin v \quad *^s \notin p \quad \ell \text{ fresh} \quad \Omega(p) \leftarrow \text{loan}^m \ell \Rightarrow \Omega'}{\Omega \vdash \&\text{mut } p \rightsquigarrow \text{borrow}^m \ell \vdash \Omega'}
\\[10pt]
\text{E-MOVE} \\
\frac{\Omega(p) \Rightarrow v \quad \{\perp, \text{loan}, \text{borrow}^r\} \notin v \quad \{*, *^s\} \notin p \quad \Omega(p) \leftarrow \perp \Rightarrow \Omega'}{\Omega \vdash \text{move } p \rightsquigarrow v \vdash \Omega'}
\\[10pt]
\text{E-ASSIGN} \\
\frac{\Omega \vdash rv \rightsquigarrow v \vdash \Omega' \quad \Omega'(p) \Rightarrow v_p \quad v_p \text{ has no outer loans} \quad x_{\text{old}} \text{ fresh} \quad \Omega'(p) \leftarrow v \Rightarrow \Omega'' \quad \Omega''' = \Omega''[x_{\text{old}} \mapsto v_p]}{\Omega \vdash p := rv \rightsquigarrow () \vdash \Omega'''}
\\[10pt]
\text{C-SHARED-BORROW} \\
\frac{\ell' \text{ fresh} \quad \text{loan}^s \{\ell \cup \vec{\ell}\} v \in \Omega \quad \Omega' = \left[\text{loan}^s \{\ell \cup \ell' \cup \vec{\ell}\} v / \text{loan}^s \{\ell \cup \vec{\ell}\} v \right] \Omega}{\Omega \vdash \text{copy borrow}^s \ell \Rightarrow \text{borrow}^s \ell' \vdash \Omega'}
\\[10pt]
\text{E-SHARED-OR-RESERVED-BORROW} \\
\frac{\Omega(p) \Rightarrow v \quad \{\perp, \text{loan}^m, \text{borrow}^r\} \notin v \quad \ell \text{ fresh} \quad \Omega' = \Omega[p \mapsto v'] \quad v' = \begin{cases} \text{loan}^s \{\vec{\ell} \cup \ell\} v'' & \text{if } v = \text{loan}^s \{\vec{\ell}\} v'' \\ \text{loan}^s \{\ell\} v & \text{otherwise} \end{cases}}{\Omega \vdash \&p \rightsquigarrow \text{borrow}^{r,s} \ell \vdash \Omega'}
\\[10pt]
\text{E-COPY} \\
\frac{\Omega(p) \Rightarrow v \quad \{\perp, \text{loan}^m, \text{borrow}^{m,r}\} \notin v \quad \Omega \vdash \text{copy } v \Rightarrow v' \vdash \Omega'}{\Omega \vdash \text{copy } p \rightsquigarrow v' \vdash \Omega'}
\\[10pt]
\text{E-MATCH} \\
\frac{\Omega \vdash p \xrightarrow{s} C[\overline{f = v}] \quad \Omega \vdash s \rightsquigarrow r \vdash \Omega'}{\Omega \vdash \text{match } p \text{ with } \dots \mid C \rightarrow s \mid \dots \rightsquigarrow r \vdash \Omega'}
\\[10pt]
\text{C-SHARED-LOAN} \\
\frac{\Omega \vdash \text{copy } v \Rightarrow v' \vdash \Omega'}{\Omega \vdash \text{copy loan}^s \{\vec{\ell}\} v \Rightarrow v' \vdash \Omega'}
\end{array}$$

Fig. 3. Selected Reduction Rules for LLBC. We omit: E-IFTHENELSE-F, tuples (similar to constructor), sequences (trivial). We also omit the handling of results – these prevent further execution and simply get carried through.

$$\begin{array}{c}
\text{R-NOT-SHARED} \\
\frac{\Omega(p) \Rightarrow v \quad v \neq \text{loan}^s \{\vec{\ell}\} v'}{\Omega(p) \xrightarrow{s} v}
\\[10pt]
\text{R-SHARED} \\
\frac{\Omega(p) \Rightarrow \text{loan}^s \{\vec{\ell}\} v}{\Omega(p) \xrightarrow{s} v}
\end{array}$$

Fig. 4. Auxiliary Judgment: Reading a Possibly Immutably-Shared Value. Rust allows matching on a value for which there are outstanding *shared* borrows; the auxiliary $\xrightarrow{s}$ read allows reading underneath a loan^s .

$$\begin{array}{c}
\text{WRITE-G} \\
\frac{p = P[x] \quad x \mapsto v_x \in \Omega \quad \Omega \vdash p(v_x) \leftarrow v \xrightarrow{g} v'_x \vdash \Omega' \quad \Omega'' = \Omega'[x \mapsto v'_x]}{\Omega[p \mapsto v] = \Omega''}
\\[10pt]
\text{W-G-SHARED-BORROW} \\
\frac{\text{loan}^s \{\ell \cup _ \} v_p \in \Omega \quad \Omega \vdash p(v_p) \leftarrow v \xrightarrow{g} v'_p \vdash \Omega' \quad \Omega'' = \left[\text{loan}^s \{\ell \cup _ \} v'_p / \text{loan}^s \{\ell \cup _ \} v_p \right] \Omega'}{\Omega \vdash (*^s p)(\text{borrow}^s \ell) \leftarrow v \xrightarrow{g} \text{borrow}^s \ell \vdash \Omega''}
\end{array}$$

Fig. 5. Auxiliary Judgment: Ghost Write. This judgment inherits all of the rules of the form W- $*$.

op reduces to v and produces updated environment Ω' . The $\rightsquigarrow$ judgment is overloaded for other syntactic categories. Finally, we write e.g. $\text{loan} \notin v$, with no arguments, to indicate that no kind of loan should appear in v ; we have similar syntactic conventions for other restrictions.

For mutable borrows (**E-MUT-BORROW**), we disallow: borrowing already-borrowed values (no loan); borrowing moved, uninitialized values (no $\perp$) or reserved borrows; and borrowing through a

shared borrow (no $*^s$ in p). This latter requirement refers to the place p , not the value v found at p : a value reachable via a shared borrow is, inevitably, shared, and therefore cannot be uniquely owned by means of a mutable borrow. If these premises are satisfied, we perform a ghost update of the environment, and mark p as loaned with identifier ℓ . The borrow evaluates to $\text{borrow}^m \ell v$, which embodies unique ownership of value v thanks to the exclusive loan ℓ .

For immutable borrows (**E-SHARED-OR-RESERVED-BORROW**), we disallow moved or uninitialized values (no $\perp$) and reserved borrows, but rule out mutable loans only: it is always legal in Rust to create another shared borrow from a value that has already been shared. The borrow evaluates to $\text{borrow}^s \ell$, a borrow without value ownership. We need to record the fact that a fresh loan has been handed out; we perform a ghost update on the environment, to either augment the loan-set of the borrowed value with ℓ , or to introduce a new loan at p to account for the fact that the value at that place is now immutably borrowed. The r, s in the conclusion indicates that the rule may produce *either* a reserved or a shared borrow; again, it is safe to ignore the “reserved” variant for now.

For moves (**E-MOVE**), we disallow moving: $\perp$, already-borrowed values (no loans), or reserved borrows. We also forbid moving *through* a dereference. The former prevents invalidating stray borrows; simply said, if a value has unterminated borrows, we cannot obtain full ownership of it in order to perform the move. The latter replicates Rust’s constraint that no moves are allowed under a borrow.

For copies (**E-COPY**), we disallow copying mutable or reserved borrows, or mutably-loaned values; we rely on an auxiliary judgment of the form $\Omega \vdash \text{copy } v \Rightarrow v' \dashv \Omega'$, meaning creating a copy of v in Ω produces v' and returns a fresh environment Ω' . This judgment behaves like the earlier **READ**, except for shared borrows. When copying a shared borrow, we automatically allocate a new loan (**C-SHARED-BORROW**) and augment the loan-set via a replacement (we use the substitution notation) – that is, we automatically perform a shared reborrow. When copying a shared loan (**C-SHARED-LOAN**), we simply copy the actual value without performing any shared-loan tracking; the ownership information that regards the old value is irrelevant for the newly-copied value.

For matches (**E-MATCH**), we peek at the enum tag via $\xRightarrow{s}$; actual transfer of ownership with moves and copies takes place in the suitable branch while executing s . We note that in general, our **READ** judgment may return values of the form loan^s : this is useful e.g., to enforce that a value is not loaned out, as in the premise of **E-MOVE**. For matches, however, we merely need to read the enum tag; for this, we automatically dereference shared loans via $\xRightarrow{s}$.

All of our rules are in an explicit style: we prefer to add premises, rather than rely on an implicit invariant by omission. For instance, we add many premises to **E-COPY** rather than rely on the fact that the copy auxiliary judgment has no rules for mutable borrows, reserved mutable borrows and mutable loans. This also guides our lazy implementation of reorganizations.

We are now ready to define the semantics of assignments (**E-ASSIGN**). We reduce rvs first, and remark that to obtain v , the various rules for the rv syntactic category must succeed. For instance, if the right-hand side is a move, then **E-MOVE** enforces all of its preconditions. This means **E-ASSIGN** operates with ownership of v , which maps to our intuition for assignments in the presence of ownership and, naturally, also corresponds to the Rust semantics. What we do enforce, however, is that the value v_p found at place p should not have any (outer) loans [Ho and Protzenko 2022c]. Overwriting a value that is currently loaned-out would violate safety; we need to rule this out. More precisely: loans may only appear behind pointer indirections; the value itself that is being overwritten may not contain any loan. The assignment rule ends by writing v at the new place (using **WRITE**). The rule for function call is identical, except it deals with binding the arguments, locals and return variable.

$\frac{\text{NOT-BORROWED} \quad \#V', V''. V[\cdot] = V'[\text{borrow}^m _ (V''[\cdot])]}{\text{not_borrowed_value } V}$	$\frac{\text{END-SHARED-OR-RESERVED-1} \quad \text{not_borrowed_value } V}{\Omega[x_1 \mapsto V[\text{borrow}^{r,s} \ell], x_2 \mapsto V'[\text{loan}^s \{ \ell \} v]] \hookrightarrow \Omega[x_1 \mapsto V[\perp], x_2 \mapsto V'[v]]}$
$\frac{\text{NOT-SHARED} \quad \#V', V''. V[\cdot] = V'[\text{loan}^s \{ _ \} (V''[\cdot])]}{\text{not_shared_value } V}$	$\frac{\text{END-SHARED-OR-RESERVED-2} \quad \text{not_borrowed_value } V \quad \ell \notin \vec{\ell}}{\Omega[x_1 \mapsto V[\text{borrow}^{r,s} \ell], x_2 \mapsto V'[\text{loan}^s \{ \ell \cup \vec{\ell} \} v]] \hookrightarrow \Omega[x_1 \mapsto V[\perp], x_2 \mapsto V'[\text{loan}^s \{ \vec{\ell} \} v]]}$
$\frac{\text{END-MUT} \quad \{ \text{loan}, \text{borrow}^r \} \notin v \quad \text{not_borrowed_value } V}{\Omega[x_1 \mapsto V[\text{borrow}^m \ell v], x_2 \mapsto V'[\text{loan}^m \ell]] \hookrightarrow \Omega[x_1 \mapsto V[\perp], x_2 \mapsto V'[v]]}$	$\frac{\text{ACTIVATE-RESERVED} \quad \{ \text{loan}, \text{borrow}^r \} \notin v \quad \text{not_shared_value } V'}{\Omega[x_1 \mapsto V[\text{borrow}^r \ell], x_2 \mapsto V'[\text{loan}^s \{ \ell \} v]] \hookrightarrow \Omega[x_1 \mapsto V[\text{borrow}^m \ell v], x_2 \mapsto V'[\text{loan}^m \ell]]}$

Fig. 6. Reorganizing Environments

One key point of **E-Assign** is that we retain the old value in the environment Ω'' , under a fresh name x_{old} not accessible to the user-written program. We call this retained value x a *ghost value*, because its only purpose is to avoid discarding useful ownership knowledge; operationally, this is memory that can be actually reclaimed since it isn't reachable anymore. Our third example at the beginning of the section leverages this fact.

3.4 Reorganizing Environments and Terminating Borrows

We now present the final conceptual portion of our operational semantics: reorganizing the environment, which we used in our earlier examples to terminate borrows. We present rules in a declarative style, to highlight the *semantics* of Rust as opposed to the *implementation* of borrow-checking. A consequence of our declarative approach is that we do not need to follow Rust's behavior to the letter; rather, we reorganize borrows in a lazy fashion, and don't terminate a borrow unless we need to get the borrowed value back. Concretely, our rules allow reorganization before and after every statement (we have elided this from Figure 3 for clarity). This has two concrete consequences. First, we ignore the drop nodes from MIR – indeed, they do not appear in Figure 1. Second, we claim that this captures a general semantics of borrows; we substantiate that claim by showing, in Section 6, how our semantics can validate a Rust program checked with Polonius, an ongoing rewrite of the borrow checker to allow for a larger class of Rust programs to be accepted.

We define reorganizing via a set of rewriting rules that operate on the environment Ω (Figure 6). Since these rules are syntactic in nature, we rely on value contexts $V[v]$, rather than our earlier semantic notions of reads, writes and ghost updates. We omit administrative rules for re-ordering environments at will. Our judgments are of the form $\Omega \hookrightarrow \Omega'$, meaning Ω may be reorganized into Ω' . We indulge in some syntax overload; whenever used on the left-hand side of $\hookrightarrow$, we understand $\Omega[x \mapsto v]$ to pattern-match on Ω to select a mapping. This considerably simplifies notation.

Our rules either render a value unusable ($\perp$), or strengthen it (borrow_m , in the case of reserved borrows). For these reasons, we demand unique ownership of the value in V via **NOT-BORROWED**; that is, we can only terminate borrows for values that are not themselves borrowed. Doing so, we precisely capture the constraints of Rust with regards to reborrows. We now review the rules.

When ending a shared borrow, we render the borrow unusable henceforth, and replace it with $\perp$. Then, two situations arise. If this is the last borrow (**END-SHARED-OR-RESERVED-1**), i.e., if the loan-set

is the singleton set $\{\ell\}$, we replace the shared loan with the previously-shared value. If there are more borrows out there (**END-SHARED-OR-RESERVED-2**), we simply decrease the loan-set of the shared loan to reflect that the borrow has been ended.

When ending a mutable borrow (**END-MUT**), we enforce that we own the value we are about to return (i.e. not loaned). The borrow then becomes unusable, and the borrowed value is returned to its rightful owner.

This high-level approach to the Rust semantics allows us to very naturally account for an oft-used Rust feature, namely two-phase borrows, which are introduced in many places when desugaring to MIR. We account for those through what we call *reserved* borrows. Reserved borrows are created just like shared borrows (**E-SHARED-OR-RESERVED-BORROW**). However, reserved borrows cannot be copied, dereferenced, or written into. Therefore, the only way to use a reserved borrow is to strengthen it into a mutable borrow, which is legal, as long as all other (shared or reserved) borrows have ended (**ACTIVATE-RESERVED**). Reserved borrows enable a variety of very common idioms [The Rust Compiler Team 2022] without resorting to more advanced desugarings.

These rules are declarative and non-ordered; in practice, our tool performs a syntax-directed reorganization guided by the various preconditions on our rules. For instance, whenever $\text{loan} \notin v$ appears as a premise, we perform a traversal of v to end whichever loans we encounter. This is another reason why we prefer the “explicit” style of rules (i.e. with copious premises): they clearly state expectations, and thus allow for a straightforward implementation.

4 SYMBOLIC ABSTRACTIONS AND FUNCTIONAL TRANSLATION

Our semantics allows us to keep track of borrows and ownership in an exact fashion. We now ask: if we adopt a modular approach and treat function calls as opaque, how much can we leverage borrows and regions to still enable precise tracking of ownership and aliasing? We answer that question with a region-centric shape analysis that abstracts away the effect of a function call on the ownership graph, via a notion of *region abstraction*. We dub the result our “symbolic semantics”; it is, obviously, less precise than our earlier concrete semantics; yet, it contains enough ownership and aliasing information that we can generate a functional translation from it. Our symbolic semantics very much resembles the concrete semantics; this time, however, we turn out attention to the region information provided by function signatures to abstract away subsets of the ownership graph.

From this section onwards, we introduce a few additional restrictions on the subset of Rust we can handle. Our concrete semantics supports loops, but our symbolic semantics does not; we *do*, however, support recursive functions. We disallow nested borrows in function *signatures*, but users can still manipulate arbitrarily nested borrows within function bodies. We also disallow instantiating a polymorphic function with a type argument that contains a borrow. Finally, we do not allow type declarations that contain borrows. We believe most of these issues can be addressed with suitable amounts of engineering; we discuss these limitations in detail in Section 7.

4.1 Symbolic Semantics by Example

Symbolic Values; Matches. A first concept we need to add to our toolkit is that of a symbolic variable; that is, a variable whose type is known, but not its value: we write $(\sigma : \tau)$. We now illustrate how symbolic variables behave, notably in the presence of *matches*, which refine our static knowledge about a symbolic variable. From here on, we make many constructions explicit so as to study a valid LLBC program; importantly, moves are now materialized.

```

1 fn f(mut o: Option<i32>){ // o ↦ (σ : Option i32)
2   let po = &mut o;      // o ↦ loanm ℓ; po ↦ borrowm ℓ (σ : Option i32)
3
4   match *po {
```

```

5  None => {                               // o ↦ loanm ℓ;   po ↦ borrowm ℓ (σ : None)
6    panic!() }
7
8  Some => {                                // o ↦ loanm ℓ;   po ↦ borrowm ℓ (Some (σ' : i32))
9    let r =
10      &mut (*po).Some.0; // o ↦ loanm ℓ;   po ↦ borrowm ℓ (Some loanm ℓ'); oref ↦ borrowm ℓ' (σ' : i32)
11    *r = 1; }}; // o ↦ loanm ℓ;   po ↦ borrowm ℓ (Some loanm ℓ'); oref ↦ borrowm ℓ' 1

```

In the example above, σ stands in for the function parameter whose concrete value is unknown at run-time; σ behaves like any other value from our previous examples, and can be borrowed mutably (line 2).

A key requirement for the soundness of our semantics is to forbid changing the enum variant of $\mathbf{o}$, while its value or one of its fields is borrowed: this disallows leftover borrows pointing to data of the wrong (previous) type. We enforce this soundness criterion as follows. Assume, for the sake of example, that the user at line 3 decides to mutate $\mathbf{o}$, e.g. by doing $\mathbf{o} = \mathbf{None}$. Our semantics for assignments looks up the symbolic value for the left-hand side of the assignment (i.e., $\mathbf{o}$), and demands that the symbolic value have no outstanding “outer” loans (we formally define this criterion in [Ho and Protzenko 2022c]). In order to satisfy this criterion, we must terminate the borrow $\mathbf{po}$ in order to obtain $\mathbf{o} \mapsto \sigma; \mathbf{po} \mapsto \perp$, which then prevents any further use of $\mathbf{po}$ – we have successfully prevented a type-incorrect usage.

At line 4, we perform a case analysis; at this stage, all we know is that the scrutinee $\mathbf{*po}$ evaluates to symbolic variable σ , of the correct type Option i32. In order to check the branches, we treat each one of them individually, in each case refining σ with a more precise value according to the constructor of the branch. Simply said, in the **None** case, we replace every occurrence of σ with **None**, and in the **Some** case, we replace every occurrence of σ with **Some** ($\sigma' : \text{i32}$), where σ' is a fresh symbolic variable.

More interesting pointer manipulations follow in the **Some** branch. We borrow the value within the option via $\mathbf{r}$, using a projector syntax inspired by MIR’s internal representation of projectors. This borrowing incurs no loss in precision in our alias tracking: because we refined σ earlier (in effect, performing a strong-update of σ), we know that *both* $\mathbf{o}$ and $\mathbf{po}$ are unusable as long as $\mathbf{r}$ lives. More specifically, and in the vein of our remark above: should the user, for the sake of example, decide to mutate via $\mathbf{po}$, e.g. to change the enum variant by doing $\mathbf{*po} = \mathbf{None}$ at line 10, our symbolic semantics would give up ownership of $\mathbf{r}$, in order to regain $\mathbf{po} \mapsto \text{borrow}^m \ell$ (Some 1), which by virtue of containing no “outer” loans would make the update valid (see 3.3, discussion of **E-Assign**).

Function Calls: Single region case. We now switch from the callee to the caller’s perspective, and turn our attention to function calls. We introduce a new concept of region abstraction to our borrow graph. An abstraction owns borrows and loans, but does so abstractly; that is, we have no aliasing information about values in an abstraction. Region abstractions allow us to retain ownership and aliasing information in the presence of function calls; they are introduced when a call takes place, upon which they assume ownership of the call’s arguments; they are terminated whenever the caller relinquishes ownership of the return value, upon which ownership flows back to the original arguments.

Before modifying the semantics from Section 3, we illustrate region abstractions with an example. We revisit our earlier **test_choose** function (Section 2).

```

1  let mut x = 0; let px = &mut x;
2  let mut y = 0; let py = &mut y; // x ↦ loanm ℓx, y ↦ loanm ℓy, px ↦ borrowm ℓx 0, py ↦ borrowm ℓy 0
3  let pz = choose(true, move px, move py);
4  // x ↦ loanm ℓx, y ↦ loanm ℓy, px ↦ ⊥, py ↦ ⊥, pz ↦ borrowm ℓr (σ : uint32),
5  // A(p) { borrowm ℓx 0, borrowm ℓy 0, loanm ℓr }
6  *pz = *pz + 1;

```

```

7 //  $x \mapsto \text{loan}^m \ell_x$ ,  $y \mapsto \text{loan}^m \ell_y$ ,  $px \mapsto \perp$ ,  $py \mapsto \perp$ ,  $pz \mapsto \text{borrow}^m \ell_r$  ( $\sigma' : \text{uint32}$ ),      step 0
8 //  $A(\rho) \{ \text{borrow}^m \ell_x 0, \text{borrow}^m \ell_y 0, \text{loan}^m \ell_r \}$ 
9 //  $x \mapsto \text{loan}^m \ell_x$ ,  $y \mapsto \text{loan}^m \ell_y$ ,  $px \mapsto \perp$ ,  $py \mapsto \perp$ ,  $pz \mapsto \perp$ ,      step 1
10 //  $A(\rho) \{ \text{borrow}^m \ell_x 0, \text{borrow}^m \ell_y 0, \sigma' \}$ 
11 //  $x \mapsto \text{loan}^m \ell_x$ ,  $y \mapsto \text{loan}^m \ell_y$ ,  $px \mapsto \perp$ ,  $py \mapsto \perp$ ,  $pz \mapsto \perp$ ,      step 2
12 //  $px' \mapsto \text{borrow}^m \ell_x \sigma_x$ ,  $py' \mapsto \text{borrow}^m \ell_y \sigma_y$ 
13 //  $x \mapsto \sigma_x$ ,  $y \mapsto \text{loan}^m \ell_y$ ,  $px \mapsto \perp$ ,  $py \mapsto \perp$ ,  $pz \mapsto \perp$ ,  $px' \mapsto \perp$ ,  $py' \mapsto \text{borrow}^m \ell_y \sigma_y$       step 3
14 assert!( $x == 1$ );

```

Up to line 2, the usual set of rules apply and yield an environment that is consistent with Section 3. Our abstract rules come in at line 3, where we are faced with a function call. We now need to abstract the call, that is, precisely capture how the function call affects the borrow graph, without looking at the definition of the function itself. To do so, we have only one piece of information at our disposal: the type of **choose**, namely $(\text{bool}, \&^p \text{mut uint32}, \&^p \text{mut uint32}) \rightarrow \&^p \text{mut uint32}$.

The type of **choose** conveys two key pieces of information: first, it *consumes* two mutable borrows in order to *produce* a fresh (abstract) return value; second, the borrows and the return value belong to the same *region* ρ . We proceed as follows. We allocate a fresh region abstraction $A(\rho)$, which owns the consumed arguments pertaining to region ρ ; in our case, $\text{borrow}^m \ell_x 0$ and $\text{borrow}^m \ell_y 0$. (In the case of multiple regions per function type, we need to project the ownership of the arguments along their respective regions; we handle this case formally in § 4.2.) We know that the return value **pz** has type $\&^p \text{mut uint32}$; furthermore, the region in the type tells us that the owner of this abstract value is the abstraction $A(\rho)$. We perform a *symbolic expansion* (also detailed in the next section) to give **pz** the shape $\text{borrow}^m \ell_r$ ($\sigma : \text{uint32}$), pointing into a $\text{loan}^m \ell_r$ for the return value owned by $A(\rho)$. We use σ to denote a “symbolic value”; such values are not statically known, and receive a special treatment during the translation. We obtain the environment at lines 4-5, where the ownership of both **px** and **py** has been transferred to the region abstraction; and where **pz** has full ownership of a value loaned from the region abstraction. Intuitively, a region abstraction is a bag containing borrows (what has been consumed) and loans (what has been produced).

At line 6, the mutation type-checks, and does not affect the abstract environment: the symbolic value σ borrowed through **pz** is simply replaced by a fresh symbolic value σ' stemming from the addition. At that stage, we cannot read from **x** since it is mutably loaned; we therefore need to reorganize the environment to make the assertion succeed. Since we do not have any precise knowledge about the aliasing relationship between **x**, **y** and **pz**, we cannot return ownership to **x** directly; we must return ownership *en masse* by terminating region $A(\rho)$. We do so by terminating the borrow for **pz**, which returns the abstract value σ' to $A(\rho)$ (step 1, lines 9-10). Now that $A(\rho)$ has no outstanding loans left, we can terminate $A(\rho)$ itself. This reintroduces in the environment borrows ℓ_x and ℓ_y with fresh values, and replaces the borrowed values they held (0 in both cases) with fresh symbolic values to account for potential modifications (lines 11-12). These borrows are ghost values, i.e. not directly accessible to the user; they once again ensure we do not lose ownership information. A final reorganization of the environment terminates ℓ_x , and makes **x** usable again (line 13).

Multiple region case. We now study a call to the **swap** function, which permutes the two components of a single tuple, located in two different regions.

$$\text{swap} : (z : (\&^\alpha \text{mut uint32}, \&^\beta \text{mut uint32})) \rightarrow (\&^\beta \text{mut uint32}, \&^\alpha \text{mut uint32})$$

We examine a call **let r = swap (move z)** in the following environment:

$$x \mapsto \text{loan}^m \ell_x, \quad y \mapsto \text{loan}^m \ell_y, \quad z \mapsto (\text{borrow}^m \ell_x 0, \text{borrow}^m \ell_y 0)$$

This time, the presence of two regions forces us to be more precise. We introduce a new notion of projector, which comes in three flavors. We use an *input borrow projector* to dispatch each component of the argument to its respective region abstraction. We use a *loan projector* to dispatch

each component of the returned value to its respective region abstraction. And we use an *output borrow projector* to determine the shape of the return value based on its type information. (We write these proj_{in} , proj_l and proj_{out} , respectively.) Doing so, we rely on *expansion rules* to destruct and name the components of various tuples as needed. Thus, the environment after the function call is:

$$\begin{aligned} & x \mapsto \text{loan}^m \ell_x, \quad y \mapsto \text{loan}^m \ell_y, \quad z \mapsto \perp, \quad r \mapsto \text{proj}_{\text{out}}(\sigma : (\&^\beta \text{mut uint32}, \&^\alpha \text{mut uint32})) \\ & A(\alpha)\{ \\ & \quad \text{proj}_{\text{in}}((\text{borrow}^m \ell_x 0, \text{borrow}^m \ell_y 0) : (\&^\alpha \text{mut uint32}, \&^\beta \text{mut uint32})) \\ & \quad \text{proj}_l(\sigma : (\&^\beta \text{mut uint32}, \&^\alpha \text{mut uint32})) \\ & \} \\ & A(\beta)\{ \\ & \quad \text{proj}_{\text{in}}((\text{borrow}^m \ell_x 0, \text{borrow}^m \ell_y 0) : (\&^\alpha \text{mut uint32}, \&^\beta \text{mut uint32})) \\ & \quad \text{proj}_l(\sigma : (\&^\beta \text{mut uint32}, \&^\alpha \text{mut uint32})) \\ & \} \end{aligned}$$

In the resulting environment, z has been consumed. The return value r needs to be decomposed using an output borrow projector, according to its type. And both region abstractions have the same content, which need to be projected along their respective regions α and β .

A first, new reorganization rule (Figure 8) allows us to refine the symbolic value σ to be a tuple (σ_l, σ_r) . Next, the two input projectors reduce based on the type: the left component in $A(\alpha)$ remains, as it belongs to the enclosing region α , and the right component reduces to $\perp$, an ignored value that does not belong to α . (The case of β is symmetrical.) The loan projector behaves similarly and retains only the components pertaining to the enclosing region; finally, the output borrow projector generates a pair of borrows pointing to the corresponding abstract values. The resulting environment is therefore as follows.

$$\begin{aligned} & x \mapsto \text{loan}^m \ell_x, \quad y \mapsto \text{loan}^m \ell_y, \quad z \mapsto \perp, \quad r \mapsto (\text{borrow}^m \ell_l \sigma_l, \text{borrow}^m \ell_r \sigma_r), \\ & A(\alpha)\{ \quad (\text{borrow}^m \ell_x 0, \perp) \quad (\perp, \text{loan}^m \ell_l) \quad \}, \\ & A(\beta)\{ \quad (\perp, \text{borrow}^m \ell_y 0) \quad (\text{loan}^m \ell_r, \perp) \quad \} \end{aligned}$$

Discussion of examples. We see our region abstractions as a form of magic wands; a function call consumes part of the memory, and returns a magic wand (the region abstraction) along with its argument (the returned value). Regaining ownership of the consumed memory requires applying the magic wand to its argument, hence surrendering access to the returned value.

Naturally, region abstractions set the stage for our functional translation: introducing an abstraction translates to a call to a *forward function*, while terminating an abstraction translates to a call to a *backward function*. But in order to get to the functional translation, we must first define region abstractions more formally.

4.2 From Concrete to Symbolic Semantics

We define our symbolic semantics as an extension of our earlier formalism, along with a new rule for function calls. First, we extend the value category to account for symbolic values, denoted σ (Figure 7), as well as our three kinds of projectors.

Following the intuition from our second example, above, we recall that the chief goal of input and loan projectors is to distribute (“project”), at function call-time, each component of a value to their corresponding abstraction region, while output borrow projectors allow destructuring the caller’s view of the return value according to its type and region. Projectors enjoy some duality: if we switch to the point of view of the callee, output and loan projectors capture the fact that the effective arguments are “on loan” from the callee’s context.

We keep the syntactic overhead to a minimum. Our rules (and our implementation) enforce strong invariants: for instance, input borrow and loan projectors may only appear within region

$v ::=$	$\Omega ::=$
...	...
$(\sigma : \tau)$	$A(\rho)\{\vec{v}\}, \Omega$
$\text{proj}_{\text{in}} v$	new region abstraction for ρ
$\text{proj}_l v$	
$\text{proj}_{\text{out}} v$	$\rho ::=$
	region identifier

Fig. 7. Abstract Semantics: Environments, Values

DECOMPOSE-TUPLE $\frac{\sigma_l, \sigma_r \text{ fresh}}{\Omega \xrightarrow{(\sigma_l, \sigma_r)} \left[((\sigma_l, \sigma_r) : (\tau_1, \tau_2)) / (\sigma : (\tau_1, \tau_2)) \right] \Omega}$	PROJ-TUPLE $\frac{}{\begin{array}{l} \Omega[A(\rho) \mapsto \text{proj}_{\text{in}, l, \text{out}}(\sigma_l, \sigma_r)] \hookrightarrow \\ \Omega[A(\rho) \mapsto (\text{proj}_{\text{in}, l, \text{out}} \sigma_l, \text{proj}_{\text{in}, l, \text{out}} \sigma_r)] \end{array}}$
PROJ-I-MUT-MATCH $\frac{}{\begin{array}{l} \Omega[A(\rho) \mapsto \text{proj}_{\text{in}}(\text{borrow}^m \ell \sigma : \&^p \text{mut } \tau)] \hookrightarrow \\ \Omega[A(\rho) \mapsto \text{borrow}^m \ell (\sigma : \tau)] \end{array}}$	PROJ-I-MUT-NO-MATCH $\frac{}{\begin{array}{l} \Omega[A(\rho) \mapsto \text{proj}_{\text{in}}(\text{borrow}^m \ell _ : \&^p \text{mut } \tau)] \hookrightarrow \\ \Omega[A(\rho) \mapsto _] \end{array}}$
PROJ-I-SHARED-MATCH $\frac{}{\begin{array}{l} \Omega[A(\rho) \mapsto \text{proj}_{\text{in}}(\text{borrow}^s \ell : \&^p \tau)] \hookrightarrow \\ \Omega[A(\rho) \mapsto \text{borrow}^s \ell] \end{array}}$	PROJ-I-SHARED-NO-MATCH $\frac{}{\begin{array}{l} \Omega[A(\rho) \mapsto \text{proj}_{\text{in}}(\text{borrow}^s \ell : \&^p \tau)] \hookrightarrow \\ \Omega[A(\rho) \mapsto _] \end{array}}$
PROJ-UNFOLD-MUT-MATCH $\frac{\sigma', \ell \text{ fresh}}{\begin{array}{l} \Omega[p \mapsto \text{proj}_{\text{out}}(\sigma : \&^p \text{mut } \tau), A(\rho) \mapsto \text{proj}_l \sigma] \hookrightarrow \\ \Omega[p \mapsto \text{borrow}^m \ell \sigma', A(\rho) \mapsto \text{loan}^m \ell] \end{array}}$	PROJ-UNFOLD-SHARED-MATCH $\frac{\sigma', \ell \text{ fresh}}{\begin{array}{l} \Omega[p \mapsto \text{proj}_{\text{out}}(\sigma : \&^p \tau), A(\rho) \mapsto \text{proj}_l \sigma] \hookrightarrow \\ \Omega[p \mapsto \text{borrow}^s \ell, A(\rho) \mapsto \text{loan}^s \{\ell\} \sigma'] \end{array}}$
PROJ-L-NO-MATCH $\frac{\& \notin \tau}{\begin{array}{l} \Omega[A(\rho) \mapsto \text{proj}_l(\sigma : \tau)] \hookrightarrow \\ \Omega[A(\rho) \mapsto _] \end{array}}$	END-ABSTRACT-MUT $\frac{\text{not_borrowed_value } V}{\begin{array}{l} \Omega[x \mapsto V[\text{borrow}^m \ell v], A(\rho) \mapsto \text{loan}^m \ell] \hookrightarrow \\ \Omega[x \mapsto V[\perp], A(\rho) \mapsto v] \end{array}}$
END-ABSTRACTION $\frac{\begin{array}{l} \text{borrows}^m(A(\rho)) = \overrightarrow{\text{borrow}^m \ell _} \\ \text{borrows}^s(A(\rho)) = \overrightarrow{\text{borrow}^s \ell'} \\ \text{loan, proj}_l \notin A(\rho).v_r \quad \vec{\sigma}' \text{ fresh} \quad \vec{x}_g, \vec{y}_g \text{ fresh} \end{array}}{\Omega(\rho), \Omega \hookrightarrow \Omega, \vec{x}_g \mapsto \overrightarrow{\text{borrow}^m \ell \vec{\sigma}'}, \vec{y}_g \mapsto \overrightarrow{\text{borrow}^s \ell'}}$	

Fig. 8. Reorganizing Environments with Abstract Values and Projectors

abstractions; and only a very restricted form of values may appear under projectors. But to keep notation lightweight, we refrain from adding extra syntactic categories. In the same spirit, we introduce some syntactic sugar. Whenever type annotations are not needed, we skip τ in $(\sigma : \tau)$. Conversely, whenever we need to access the type of a value (to make a region apparent), we allow $(v : \tau)$ as a convenient way to bind the type. Finally, we start leveraging region annotations in borrow types, and introduce a new form of path $A(\rho) \mapsto$ to select an element from an abstraction.

The new rewriting rules capture the behavior demonstrated with earlier examples. A symbolic value may be decomposed structurally (**DECOMPOSE-TUPLE**); we use a substitution notation to indicate that there may be several occurrences of σ , and all of them must be substituted at the same time. (Our second example showcased this situation.) We tack the value σ being destructured and the pattern used for that purpose (σ_l, σ_r) onto the arrow, so that they can be conveniently recalled to generate a suitable let-binding in the functional translation. Symmetrically, any kind of projector descends along the structure of the symbolic value (**PROJ-TUPLE**).

Input borrow projectors, once confronted with a borrow value, either discard it (as in either **PROJ-I-MUT-NO-MATCH** or **PROJ-I-SHARED-NO-MATCH**) or retain it (as in either **PROJ-I-MUT-MATCH** or **PROJ-I-SHARED-MATCH**). As we forbid nested borrows for now, projections stop upon the first borrow they encounter; if the projector retains the borrow, we keep the whole borrowed value.

Loan and output borrow projectors reduce in lockstep: if the environment contains an output borrow projector and a region abstraction contains a corresponding loan projector over the same symbolic value σ , then we may turn them into a borrow and a loan, respectively (**PROJ-UNFOLD-MUT-MATCH**, **PROJ-UNFOLD-SHARED-MATCH**). To give back ownership of a return value to the abstraction, **END-ABSTRACT-MUT** folds a mutable borrow back into an abstraction. Ending a region abstraction itself is done via **END-ABSTRACTION**; we use the borrow notation to collect all the borrows, at once from all the values in the abstraction. We require two things. First, that the abstraction has no outstanding loans, i.e. the contents of the abstraction are either symbolic values or borrows. Second, that the abstraction contains no loan projectors. This latter precondition avoids dangling output borrow projectors once the abstraction has disappeared. Terminating the region abstraction returns ownership of the borrows, with fresh symbolic values, to fresh (ghost) variables.

4.3 From Symbolic Semantics to Functional Code

At last, we explain how AENEAS, using the symbolic semantics, generates a pure translation of the original LLBC program. We make several hypotheses at this stage; beyond the restrictions we already mentioned, we now assume every disjunction in the control-flow is in terminal position. This is a strong restriction, which in practice requires duplicating the continuation of conditionals and matches. This is also a well-understood problem, known as “computing a join” in abstract interpretation for shape analysis domains [Rival 2011], or the “merge problem” in Mezzo [Protzenko 2014]. Coupled with the fact that problem space is highly constrained by Rust’s lifetime discipline, we are confident that this can be addressed systematically and predictably.

The rules are in Figure 9, where T- stands for translation; they describe a process in which we traverse the source program in a forward fashion, simultaneously updating our symbolic environment and generating pure λ -terms. Our final judgment operates over LLBC statements s and takes two forms. For statements that are in terminal position, we write $M, \Omega \vdash s \Downarrow e$, meaning statement s compiles down to pure expression e in environment Ω and translation meta-data M . But for statements that are not in terminal position (i.e., on the left-hand side of a semicolon), we are faced with the usual mismatch between statement-based languages and let/expression-based languages. We solve the issue by allowing expressions to contain a hole, to receive a continuation – we write $E[\cdot]$. Our judgement for non-terminal statements is thus of the form $M, \Omega \vdash s \Downarrow E[\cdot] \dashv M', \Omega'$ – we note that this form produces an updated environment M', Ω' to allow chaining with subsequent statements. In practice, our implementation uses continuation-passing style to keep things readable, as opposed to an AST definition with holes for our target language. In both cases, we let $\Downarrow$ be either $\Downarrow$, for translating a forward function, or $\Uparrow^\rho$, for translating the backward function associated to region ρ . We spare the reader the grammar of expressions e , which is a standard lambda-calculus; suffices to say that we use $\leftarrow$ to denote the monadic bind operator, as in Section 2; monadic returns appear as `ret`. The synthesis of variables is captured

by the rules PURE-^* ; as we alluded to earlier, our translation never encounters, nor produces, a source variable x ; rather, they structurally visit a symbolic value and map source *symbolic* variables to variables in the target λ -calculus (PURE-SYMB). Naturally, in practice, we use heuristics to pick sensible names for the symbolic variables, thus guaranteeing that the output of our translation is readable. Conversion of types from source to target is almost the identity, except for $\text{Box } \tau$, $\&^\rho \text{ mut } \tau$ and $\&^\tau$ which become τ , consistently with PURE-BOX , PURE-MUT-BORROW and $\text{PURE-SHARED-BORROW}$.

Another insight about our rules: in order to make progress, we may synthesize fresh bindings at any time via a reorganization. For instance, if a symbolic value with a tuple type is refined into the tuple of its components (DECOMPOSE-TUPLE), we need to mirror this fact in the generated program (T-DESTRUCT). We use a degenerate judgment of the form $M, \Omega \vdash \emptyset \Downarrow \dots$, which appears in T-DESTRUCT , T-REORG-ANYTIME and T-CALL-BACKWARD .

The rest of the rules leverage our earlier concepts of region abstractions, projections, and symbolic environments to precisely capture the relationship between a function body and its parameters (callee), or a function application and its arguments (caller). The rules for synthesis generally apply both for generating forward and backward functions, with the exception of T-RETURN and T-FUN .

We adopt the perspective of the caller and begin with function calls. In T-CALL-FORWARD , we follow the procedure outlined in our earlier examples. First, we allocate a fresh σ_r to stand in for the return value of the call; we have one abstraction region per region in the function type. Each abstraction region gains ownership of the relevant part of the function arguments ($\text{proj}_{\text{in}} \vec{v}$), and loans out whichever part of the symbolic return value originates from that region ($\text{proj}_l \sigma_r$). These abstractions augment the symbolic environment, along with an output projector for the return value σ_r . We synthesize a monadic bind which introduces σ_r in the generated program, and record in the meta-data M that this call happened with effective arguments $\vec{v}$. Rule T-CALL-BACKWARD is not syntax-directed and may happen at any time; in practice, we apply it lazily. We wish to terminate region ρ , associated to an earlier function call found in M . We require a successful application of END-ABSTRACTION , so as to terminate abstraction region $A(\rho)$. This returns ownership to us (the caller) of various borrows of symbolic values $\vec{\sigma}'$. We thus need to synthesize a call to the backward function for region ρ of f , in order to compute in the translated code what is the value of $\vec{\sigma}'$. The backward function receives the *original* effective arguments to f (found in M), and the symbolic values that stands for the terminated projector loans from $A(\rho)$.

We now switch to the perspective of the *callee* and study function definitions. We once again rely on region abstractions to explain the ownership relationship between arguments and function body. In T-FUN-FORWARD , we synthesize f_{fwd} . In our initial environment, we have one abstraction per region in the type of f ; each abstraction region owns the parts of each argument that are along region ρ_i ; we execute the body in an environment where the formal arguments $\vec{x}_{\text{arg}}$ are each wrapped in an output projector. The backward functions are synthesized in a similar way, except with extra arguments.

Matches are delicate, and come in two flavors. We remark that our matches are made up of non-nested, constructor patterns; by the time we examine Rust's internal MIR language, nested patterns have already been desugared.

If our symbolic environment has enough static knowledge to determine which particular branch of the data type we are in (T-MATCH-CONCRETE), we do not bother with generating a trivial match and simply generate code for the corresponding branch. If the scrutinee is not known statically, then it is a symbolic variable (T-MATCH-SYMBOLIC). For each branch i , we effectively perform a strong update of σ , replacing it with a constructor value whose fields are themselves fresh symbolic variables (the $\vec{\sigma}_i$). The same $\vec{\sigma}_i$ appear in the translated code (our target lambda calculus *can* bind arguments to constructors), thus preserving proper lexical binding in the generated code. We remark that refining

into a constructor value is important for soundness – lacking this precise substructural tracking, we would lose precision in our borrow checking and would allow type-incorrect programs.

We finally explain the rules for returning. In **T-RETURN-FORWARD**, the value found in the special return variable dictates what we return, so long as we have ownership of this value. In **T-RETURN-BACKWARD**, our goal is now to map sub-parts of v_{ret} back to their original locations, using our region abstraction analysis. We “symbolize” v_{ret} using an auxiliary function, `elided`; intuitively, `sym` replaces the sub-parts of v_{ret} (what we symbolically know about the callee’s return value x_{ret} , so far) with matching sub-parts from σ_{ret} (the caller-updated return value, bound in **T-FUN-BACKWARD**, provided at call site in **T-CALL-BACKWARD**). For instance:

$$\text{sym}(\alpha, (\sigma_l, \sigma_r), (\text{borrow}^m \ell \ v : \&^\alpha \text{mut } \tau, \text{borrow}^m \ell' \ v' : \&^\beta \text{mut } \tau')) = (\text{borrow}^m \ell \ \sigma_l, \perp)$$

We then allow reorganizing the environment, so as to allow destructuring σ_{ret} suitably; this may also trigger calls to **T-CALL-BACKWARD**, as is the case for `list_nth_back`. If the region abstraction $A(\rho)$ is fully closed, the values it now holds determine the tuple we pass to the monadic return.

Those are arguably the most important rules: we illustrate them with **choose** (Section 2). In **choose_back**, σ_{ret} corresponds to the `ret` argument. In the **true** branch, v_{ret} is $\text{borrow}^m \ell_x \ \sigma_x$. Calling `sym` produces $\text{borrow}^m \ell_x \ \sigma_{\text{ret}}$. We end ℓ_x , and thus propagate σ_{ret} back to the loan for ℓ_x . Besides, ending ℓ_y gives back the unchanged σ_y and thus closes the region abstraction $A(\alpha)$, which now contains $\{\sigma_{\text{ret}}, \sigma_y\}$; we return the symbolic values in the exact same order, that is, **(ret, y)**.

5 IMPLEMENTING AENEAS

Our implementation is written in a mixture of Rust and OCaml. A first tool, dubbed **CHARON**, performs the translation from Rust’s MIR internal representation to LLBC. Concretely, **CHARON** is a Rust compiler plugin that performs a large amount of mundane, tedious tasks, such as: computing a dependency graph, reordering definitions, grouping mutually recursive definitions together, reconstructing data type creation, and generally getting rid of the idioms that are definitely too low-level for LLBC (Section 3.1). Once this is done, **CHARON** dumps a JSON file to disk containing the LLBC AST. We plan to switch to a more efficient binary format in the future. **CHARON** totals 9.5 kLoC (lines of code, excluding whitespace and comments). **CHARON** lives as a separate project because we believe it has an existence of its own outside of **AENEAS**; we could easily see other projects re-using a lot of the engineering work we performed in order to share the implementation burden. We hope to present **CHARON** to several other tool authors in the near future.

AENEAS picks up the **CHARON**-generated AST, and implements the transformations described in Section 3 and Section 4. Practically speaking, we have a single interpreter that runs in two modes, either concrete or symbolic. The former produces a final value, if running a closed term; the latter produces a translated program. We currently extract to F^* , and have a Coq backend in the works, along with plans for an HOL4 and a Lean backend. Effectively, our symbolic interpreter acts as a borrow checker for Rust programs using our semantic notion of borrows; we plan to investigate whether we can isolate this checker to validate, e.g., bare C programs that would fit within our admissible subset. We have written **AENEAS** in OCaml, a language much better suited to the manipulation of ASTs than Rust. The implementation of **AENEAS** totals 13.5kLoC. Of those, 6kLoC are for the interpreters, and 4kLoC for the translation and extraction. The rest contains library functions.

The implementation is, naturally, trusted. However, we have taken extraordinary care to ensure that it is trustworthy. Notably, after every application of one of the rules, we verify a large amount of invariants, such as: the environment is well-typed; borrows are consistent; shared values don’t contain a mutable loan, etc. In practice, those invariants are extremely tight, and have led to the great level of detail that our rules exhibit. Should one turn off those invariant checks, the whole

$\frac{\text{PURE-MUT-BORROW} \quad \Omega \vdash v \uparrow e}{\Omega \vdash \text{borrow}^m \ell \uparrow e}$	$\frac{\text{PURE-CONST}}{\Omega \vdash n_{i32} \uparrow n_{i32}}$	$\frac{\text{PURE-TUPLE} \quad \Omega \vdash \vec{v} \uparrow \vec{e}}{\Omega \vdash (\vec{v}) \uparrow (\vec{e})}$	$\frac{\text{PURE-SYMB}}{\Omega \vdash \sigma \uparrow \sigma}$	$\frac{\text{PURE-BOX} \quad \Omega \vdash v \uparrow e}{\Omega \vdash \text{Box } v \uparrow e}$
$\frac{\text{PURE-SHARED-BORROW} \quad \text{loan}^s \{ \vec{\ell} \} v \in \Omega \quad \ell \in \vec{\ell} \quad \Omega \vdash v \uparrow e}{\Omega \vdash \text{borrow}^s \ell \uparrow e}$	$\frac{\text{T-DESTRUCT} \quad \vec{\sigma}' \text{ fresh} \quad M, \Omega \xrightarrow{(\vec{\sigma}')} M', \Omega'}{M, \Omega \vdash \emptyset \uparrow \text{let } (\vec{\sigma}') = \sigma \text{ in } [\cdot] \vdash M', \Omega''}$	$\frac{\text{T-IFTHENELSE} \quad M, \Omega \vdash \text{op} \uparrow \sigma \vdash M', \Omega' \quad \Omega' \xrightarrow{\text{true}} \Omega_1 \quad \Omega' \xrightarrow{\text{false}} \Omega_2 \quad M', \Omega_1 \vdash s_1 \uparrow e_1 \quad M', \Omega_2 \vdash s_2 \uparrow e_2}{M, \Omega \vdash \text{if } \text{op} \text{ then } s_1 \text{ else } s_2 \uparrow \text{if } \sigma \text{ then } e_1 \text{ else } e_2}$	$\frac{\text{T-RETURN-FORWARD} \quad \Omega_i \vdash x_{\text{local},i} := \perp \rightsquigarrow () \vdash \Omega_{i+1} \quad \Omega_n(x_{\text{ret}}) \Rightarrow v \quad \Omega_n \vdash v \downarrow e \quad \{ \text{loan}, \perp, \text{borrow}^r \} \notin v}{M, \Omega_0 \vdash \text{return} \downarrow e}$	$\frac{\text{T-SEQ} \quad M, \Omega \vdash s_1 \uparrow E[\cdot] \vdash M', \Omega' \quad M', \Omega' \vdash s_2 \uparrow e \vdash M'', \Omega''}{M, \Omega \vdash s_1; s_2 \uparrow E[e] \vdash M'', \Omega''}$
$\frac{\text{T-REORG-ANYTIME} \quad M, \Omega \vdash \emptyset \uparrow E[\cdot] \vdash M', \Omega' \quad M', \Omega' \vdash s \uparrow E'[\cdot] \vdash M'', \Omega''}{M, \Omega \vdash s \uparrow E[E'[\cdot]] \vdash M'', \Omega''}$	$\frac{\text{T-CALL-FORWARD} \quad A(\rho_i) = \left\{ \overrightarrow{\text{proj}_{\text{in}} v}, \text{proj}_{\text{out}} \sigma_{\text{ret}} \right\} \quad (\sigma : \tau) \in \vec{v} \text{ implies } \& \neq \tau \quad \vec{\rho} \text{ fresh} \quad \sigma_{\text{ret}} \text{ fresh} \quad \Omega \vdash v_i \uparrow e_i \quad \Omega, \overrightarrow{A(\rho)} \vdash p := \text{proj}_{\text{out}} \sigma_{\text{ret}} \rightsquigarrow () \vdash \Omega'}{M, \Omega \vdash p := f(\vec{\rho}) \vec{v} \uparrow \sigma_{\text{ret}} \leftarrow f_{\text{fwd}} \vec{e}; [\cdot] \vdash (f(\vec{\rho}) \vec{e}), M; \Omega'}$	$\frac{\text{T-CALL-BACKWARD} \quad \Omega = A(\rho), \Omega' \quad M = f(\dots, \rho, \dots) \vec{v}, M' \quad \Omega \vdash A(\rho).v_{\text{ret}} \uparrow e_{\text{ret}} \quad A(\rho), \Omega' \hookrightarrow \Omega'', x_g \mapsto \text{borrow}^m \ell \sigma', y_g \mapsto \dots \text{ via } \text{END-ABSTRACTION}}{M, \Omega \vdash \emptyset \uparrow (\vec{\sigma}') \leftarrow f_{\text{back}(\rho)} \vec{e} e_{\text{ret}}; [\cdot] \vdash M', \Omega''}$	$\frac{\text{T-FUN-FORWARD} \quad A(\rho_i) = \left\{ \overrightarrow{\text{proj}_{\text{in}} (\sigma : \tau)} \right\} \quad \vec{\sigma} \text{ fresh} \quad \emptyset, \Omega \vdash s \downarrow e \quad \Omega = \overrightarrow{A(\rho)}, x \mapsto \text{proj}_{\text{out}} \sigma, \vec{x}_{\text{local}} \mapsto \vec{\perp}, x_{\text{ret}} \mapsto \perp}{\text{fn } f(\vec{\rho}) (\vec{x} : \vec{\tau}) (\vec{x}_{\text{local}} : \vec{\tau}_{\text{local}}) (x_{\text{ret}} : \tau_{\text{ret}}) = s \downarrow \text{def } f_{\text{fwd}} = \lambda(\vec{\sigma} : \vec{\tau}).e}$	$\frac{\text{T-FUN-BACKWARD} \quad A(\rho_i) = \left\{ \overrightarrow{\text{proj}_{\text{in}} (\sigma : \tau)} \right\} \quad \vec{\sigma} \text{ fresh} \quad \emptyset, \Omega \vdash s \uparrow^\rho e \quad \Omega = \overrightarrow{A(\rho)}, x \mapsto \text{proj}_{\text{out}} \sigma, \vec{x}_{\text{local}} \mapsto \vec{\perp}, x_{\text{ret}} \mapsto \perp}{\text{fn } f(\vec{\rho}) (\vec{x} : \vec{\tau}) (\vec{x}_{\text{local}} : \vec{\tau}_{\text{local}}) (x_{\text{ret}} : \tau_{\text{ret}}) = s \uparrow^\rho \text{def } f_{\text{back}(\rho)} = \lambda(\vec{\sigma} : \vec{\tau})(\sigma_{\text{ret}} : \text{proj}_{\rho} \tau_{\text{ret}}).e}$
$\frac{\text{T-RETURN-BACKWARD} \quad \Omega_i \vdash x_{\text{local},i} := \perp \rightsquigarrow () \vdash \Omega_{i+1} \quad \Omega_n(x_{\text{ret}}) \Rightarrow v_{\text{ret}} \quad \{ \text{loan}, \perp, \text{borrow}^r \} \notin v_{\text{ret}} \quad \Omega' = \Omega_n[x_{\text{ret}} \mapsto \text{sym}(\rho, \sigma_{\text{ret}}, v_{\text{ret}})] \quad \Omega' \vdash \emptyset \uparrow^\rho E[\cdot] \vdash \Omega'' \quad \Omega''[A(\rho)] = \{ \vec{v} \} \quad \{ \text{loan}, \text{proj} \} \notin \vec{v} \quad \Omega \vdash \vec{v} \uparrow \vec{e}}{\Omega_0 \vdash \text{return} \uparrow^\rho E[\text{ret}(\vec{e})]}$	$\frac{\text{T-MATCH-CONCRETE} \quad \Omega(p) \xrightarrow{s} C[f = v]}{M, \Omega \vdash s \uparrow e}$	$\frac{\text{T-MATCH-SYMBOLIC} \quad \Omega(p) \xrightarrow{s} (\sigma : t \vec{\tau}) \quad \text{type } t \vec{a} = C[\vec{f} : \vec{\tau}_f] \quad \vec{\sigma}_i \text{ fresh} \quad M, \left[(C_i[\vec{f}_i : \vec{\sigma}_i] : t \vec{\tau}) \middle/ (\sigma : t \vec{\tau}) \right] \Omega \vdash s_i \uparrow e_i}{M, \Omega \vdash \text{match } p \text{ with } \overrightarrow{C} \rightarrow s \uparrow \text{match } \sigma \text{ with } \overrightarrow{C} \vec{\sigma} \rightarrow e}$	$\frac{\text{T-MATCH-CONCRETE} \quad \Omega(p) \xrightarrow{s} C[f = v]}{M, \Omega \vdash s \uparrow e}$	$\frac{\text{T-MATCH-SYMBOLIC} \quad \Omega(p) \xrightarrow{s} (\sigma : t \vec{\tau}) \quad \text{type } t \vec{a} = C[\vec{f} : \vec{\tau}_f] \quad \vec{\sigma}_i \text{ fresh} \quad M, \left[(C_i[\vec{f}_i : \vec{\sigma}_i] : t \vec{\tau}) \middle/ (\sigma : t \vec{\tau}) \right] \Omega \vdash s_i \uparrow e_i}{M, \Omega \vdash \text{match } p \text{ with } \overrightarrow{C} \rightarrow s \uparrow \text{match } \sigma \text{ with } \overrightarrow{C} \vec{\sigma} \rightarrow e}$

Fig. 9. Functional Translation via our Symbolic Semantics

Table 1. Comparison of Verification Frameworks Targeting Safe Rust

Project	General borrows	Return borrows	Loops	Closures, traits	Termination	I/O	Borrow check.	Extrinsic	Executable
AENEAS	✓	✓	-	-	✓	✓	✓	✓	✓
Electrolysis [Ullrich 2016]	-	-	✓	✓	✓	-	✓	✓	✓
Creusot [Denis et al. 2021]	✓	✓	✓	✓	-	-	-	-	-
Prusti [Wolff et al. 2021]	✓	✓	✓	✓	-	-	-	-	-

General borrows: allows arbitrarily nested borrows and reborrows in function bodies; Electrolysis supports a very restricted subset of such operations. *Return borrows*: functions can return borrows; Electrolysis supports a very restricted subset of such functions. *Loops*: supports loops. *Closures, traits*: supports function pointers, closures and traits. *Termination*: allows reasoning about termination. *I/O*: allows reasoning about external state, such as I/O. *Borrow check.*: the framework doesn't trust nor need the Rust borrow checker; Prusti extracts the lifetime information computed by the Rust borrow checker, then checks that those lifetimes are correct. *Extrinsic*: the framework allows extrinsic proofs rather than intrinsic proofs. *Executable*: the framework generates an executable translation of the Rust program.

CHARON + AENEAS invocation becomes almost instantaneous, as opposed to a few seconds per file when constantly checking invariants. In addition to those 13.5kLoC, we have 6kLoC of comments; our implementation is truly written with great care. Curious readers can find both CHARON and AENEAS in the supplementary material.

Supported AENEAS Features. AENEAS follows the design philosophy that no annotations should be added to the Rust code; therefore, we have devised a few concrete mechanisms to make integration of AENEAS-generated code easier.

Whenever translating a recursive function, AENEAS emits a **decreases** annotation in the generated F* code. The annotation refers to a yet-to-be-defined lemma that proves semantic termination. This allows for a natural style, wherein the user invokes AENEAS to generate e.g. `Foo.fst`; the file references `Foo.Lemmas.fst`, which is then filled out and maintained by the user. For Coq, we intend to rely on a fuel parameter controlled by the user using a similar fashion.

We also provide support for interacting with the external world. Users of AENEAS can choose to mark some modules in a given crate as “opaque”; the resulting functions appear as an interface file only, meaning that they are *assumed*. The user can then provide a hand-maintained library of lemmas that characterize the behavior of these external functions. We handle external dependencies in a similar fashion, generating an interface which contains exactly those external functions and types which are called from within the crate. This relieves the tool authors from having to maintain wrappers for the entire Rust standard library.

Finally, we provide support for enhancing our working monad with an external world. That is, rather than working in the error monad, we generate code for a combined world and error monad. Combined with the opaque feature, this allows the user to modularly state their assumptions about the world, and gives us in practice a lightweight monadic effect system.

6 CASE STUDIES

Hash Table. To assess the efficiency of AENEAS as a verification platform, we study a resizing hash table equipped with `insert`, `get` (immutable lookup), `get_mut` (mutable lookup) and `remove`. Each bucket is a linked list; `insert` replaces the existing binding, if any; resizing is automatic once a certain threshold is reached. Our table is polymorphic in the type of the stored elements, but right now, keys have a fixed type `usize` (the proofs do not rely on that). We plan to make the implementation generic over the key type once we support traits.

We prove that our Rust hash table functionally behaves like a map. Most of the challenges revolve around resizing after the threshold has been reached, which involves reasoning about arithmetic overflow, mutable borrows for each given slot, and element moves to avoid copies. We establish several invariants about buckets and entries, then alternate between a semi-structural view (a list of lists) and a high-level view (an associative list). Our insertion lemma is as follows:

```

1  val hash_map_insert_fwd_lem (#t : Type) (self : hash_map_t t) (key : usize) (value : t) :
2    Lemma (requires (hash_map_t_inv self)) (ensures (
3      match hash_map_insert_fwd t self key value with
4      | Fail -> (* We fail only if: *)
5        None? (find_s self key) /\ (* the key is not already in the map *)
6        size_s self = usize_max (* and we can't increment `num_entries` *)
7      | Return hm' -> (* In case of success: *)
8        hash_map_t_inv hm' /\ (* The invariant is preserved *)
9        find_s hm' key == Some value /\ (* [key] maps to [value] *)
10       (forall k'. k' <> key ==> find_s hm' k' == find_s self k') /\ (* Other bindings unchanged *)
11       (match find_s self key with (* The size is incremented, iff we inserted a new key *)
12       | None -> size_s hm' = size_s self + 1
13       | Some _ -> size_s hm' = size_s self)))

```

By virtue of working with a (translated) pure program, we were able to focus on the *functional* behavior of the hash table and the important proof obligations, such as the absence of arithmetic overflows, rather than memory reasoning. Furthermore, the functional translation allowed us to prove the specifications in an *extrinsic* style, where we establish lemmas about the behavior of existing functions, rather than in an *intrinsic* style, where we sprinkle the Rust code with assertions, calls to auxiliary lemmas, and possibly materialize extra variables to aid reasoning. We find that the extrinsic style not only yields a modular proof, but also allows the control-flow of the proof to refine the control-flow of the code. For instance, `insert_no_resize` exhibits two logically different behaviors (key is present or not), even though the code does not branch. Rather than materialize two run-time code paths with different assertions, we perform the inversion in the proof only.

The proofs took a total of 4 person-days, for an implementation of 201 LoC without blanks and comments. We were hindered by some design choices of F^* , which generates proof obligations for the Z3 SMT solver; this mode of operation is well-suited to intrinsic reasoning, but there is right now no way for the user to have an interactive proof context like Coq. We are eager to investigate the usability of AENEAS when coupled with backends that rely on true interaction with tactics like Coq, and appear better suited for this kind of extrinsic, functional proofs.

To the best of our knowledge, our implementation is the first verified hash table in Rust. To obtain points of comparison, we therefore asked other non-Rust verification experts or tool authors to estimate the effort to verify a hash table using their respective frameworks. VST's hash table exercise [Andrew W Appel 2021] can be reasonably completed by students in three days; this is, however, a non-resizing hash table. A hash table verified using CFML [Pottier 2017] required a week of work to establish functional correctness; the original code, however, is in OCaml, which is a higher-level language than C. Based on these comparisons, we conclude that AENEAS is very competitive with other, more established verification frameworks.

A final point of comparison is with other data structures in Low^* , the subset of F^* that compiles to C. A recent paper [Ho et al. 2021] verifies an imperative map using an associative linked list. The authors reveal that this required several weeks of full-time work, so we can confidently claim that we vastly improve the state of the art for verifying low-level programs in F^* .

Non-Lexical Lifetimes (Polonius). We recently implemented a B ϵ -tree [Bender et al. 2015], and ran it through AENEAS; our verification efforts are ongoing. In the process of doing so, we bumped

into a limitation of the current Rust borrow checking algorithm; interestingly, this limitation does not appear with AENEAS, owing to our semantic checking of borrows.

The function below exhibits what is known as a “non-linear” lifetime, in which the borrow for `hd` must be terminated in the `then` branch in order to regain full ownership of `ls`. The existing borrow checker of Rust cannot account for this usage pattern, but an ongoing rewrite of the borrow-checker, called Polonius, can. For AENEAS, there is no difference: our precise semantics of borrows accounts of this use-case without trouble.

```
1 fn get_suffix_at_x<'a>(ls: &'a mut List<u32>, x: u32) -> &'a mut List<u32> {
2     match ls {
3         Nil => { ls }
4         Cons(hd, tl) => { // first mutable borrow occurs here
5             if *hd == x { ls // second mutable borrow occurs here
6                 } else { get_suffix_at_x(tl, x) } } }
```

I/O and External Dependencies. Real-world applications rely on external libraries and often need to interact with the external world through I/O or sockets. We elegantly model interaction with the outside environment using opaque modules, and a state type that combines memory, IO and the outside world. We set out to serialize our earlier hash table to the disk. To account for this, we author `serialize` and `deserialize` functions in a separate opaque module outside of the scope of verification. We mark the module as opaque, meaning AENEAS generates the following declarations.

```
1 type state : Type
2 val deserialize_fwd : state -> result (state & hash_map_t u64)
3 val serialize_fwd : hash_map_t u64 -> state -> result (state & unit)
```

We then hand-write salient lemmas about the two functions, e.g. serialization is the inverse of parsing; because our monad now talks about the outside state, we can precisely model the interaction of our parser/serializers with the outside world.

7 FUTURE WORK; RELATED WORK; CONCLUSION

Future work. We have admitted many of AENEAS’ current limitations through this paper. We plan to address loops and disjunctions in the control-flow as soon as possible, using the techniques we referenced earlier. Doing so should bring us closer to feature-parity with Creusot and Prusti. Next are traits, our Coq backend, and large-scale use-cases. We believe all of the above to be engineering tasks; the semantic insights are in this paper.

Our formalization provides a precise semantics of ownership in Rust; as we alluded to earlier, we can explain not only extensions of the Rust borrow-checker (Polonius), but trickier programs that Polonius cannot yet account for. Our next unit of work is a proof of soundness of our semantics and translation, possibly against Stacked Borrows and/or RustBelt. Doing so would establish AENEAS as an alternate borrow checker for Rust, possibly informing future evolutions. A final unit of future work is to strengthen the CHARON tool; many authors of Rust verification tools seem to re-implement comparable compiler plugins. Sharing engineering efforts can only benefit the wider community.

Related work. Electrolysis [Ullrich 2016] most resembles AENEAS. The tool translates Rust programs to a pure lambda-calculus, then targets the Lean [Moura et al. 2015] proof assistant. It relies on lenses to model mutable borrows, and as such comes with severe restrictions (Table 1); for instance, functions may only return borrows to their first argument. Electrolysis does not come with a formal model, and thus does not make a case for semantic correctness. As such, it resembles a very pragmatic “transpiler” rather than a compiler; for instance, traits map to type classes, because they, at a high-level, work in a similar fashion.

RustBelt [Jung et al. 2017] targets a different problem than AENEAS: proving the soundness of Rust’s type system, and proving the correctness of unsafe Rust programs using the Iris framework [Jung et al. 2018]. RustBelt is an impressive framework and allows composing safe code with unsafe code using a notion of semantic typing. We see RustBelt as the exact complement of AENEAS: RustBelt allows proving fiendishly difficult, small pieces of unsafe Rust code, while AENEAS allows reasoning about large amounts of safe Rust, without resorting to a full-fledged framework like Iris.

RustHorn [Matsushita et al. 2020] uses a device called *prophecy variables* to generate a pure, non-executable logical encoding of Rust programs. The original paper contains a non-mechanized proof that their logical encoding is sound with respects to a memory-based semantics of the original Rust program. RustHornBelt [Matsushita et al. 2022] uses the RustBelt framework to mechanically prove the soundness of the RustHorn-style logical encoding. We plan to investigate using this style of proof to mechanically establish the soundness of our ownership-centric semantics.

Creusot [Denis et al. 2021] is a tool that builds on RustHornBelt to generate proof obligations that can then be discharged to SMT; the authors have a proof of soundness formalized in Coq for a simplified model of MIR. Their design chooses *automated, intrinsic* proofs: they introduce an annotation language for specifications, wrap a large part of the standard library in it, then rely on requires/ensures clauses and annotations to perform the proofs. This style emphasizes a logical encoding as opposed to an executable specification, and a closed-world approach where the verified code cannot naturally be integrated as part of, e.g., a large Coq development. The advantage of this style is that they can easily require annotations for, e.g., loop invariants.

Prusti’s frontend [Astrauskas et al. 2019] is very similar to Creusot’s. The tool uses Rust’s type system to guide the application of rules in Viper [Juhász et al. 2014], which means they rely on the Rust borrow checker for lifetime inference but do not need to trust its results. Doing so, they automate the application of memory reasoning rules and thus avoid general-purpose memory proof search. Creusot, however, by virtue of its dedicated encoding that directly leverages lifetime information, appears to offer better verification performance than the Prusti frontend for the general-purpose Viper tool [Denis et al. 2021, §5.3].

Cogent [Amani et al. 2016; O’Connor et al. 2021] is a domain-specific language equipped with a linear type system. The Cogent compiler produces: C code; a high-level Isabelle/HOL specification; and a proof of refinement from the former to the latter. By virtue of producing an Isabelle/HOL specification, Cogent seamlessly composes with existing developments in that language, and can thus be integrated into a larger project, something AENEAS also enables. However, unlike AENEAS, the Cogent compiler does not need to be trusted since it produces a proof of translation correctness for each compilation run. We also remark that the linear type system of Cogent is significantly less expressive than Rust’s; notably, Cogent does not seem to allow an equivalent of mutable borrows.

Stacked Borrows [Jung et al. 2019] give a semantics to the notion of borrows in Rust, but sets out to achieve different goals than Aeneas: namely, to provide a set of rules that Rust developers can follow and validate their code against when writing unsafe code. The work comes with an extensive evaluation, which establishes both that the tool can detect incorrect uses (bugs were found), and that it can prove that some optimizations written using unsafe code are correct. This work adopts a very low-level view of memory, and it is unclear whether it can be used productively at the scale that we envision for AENEAS. The value of the work, however, lies in its precise, memory-based semantics of borrows; we are evaluating the feasibility of proving our semantics against it.

Table 1 compares some of the tools above to AENEAS. If anything, the table reveals that each tool adopts a unique stance on what kind of programs they aim to verify, and with what kind of toolchain. For AENEAS, the stance is as follows: we target safe Rust programs, and we believe in extrinsic reasoning. Doing so, we hit what we believe is a “sweet spot”, where the functional encoding is lightweight and accessible, and where proof engineers can be most productive.

ACKNOWLEDGMENTS

We are very grateful to Aymeric Fromherz who bravely proofread this paper repeatedly at undue hours, and gave many useful remarks and feedback. We warmly thank Xavier Denis and Jacques-Henri Jourdan for insightful discussions and useful advice throughout the design of AENEAS. We also thank Chris Hawblitzel and Andrea Lattuada for a thorough walk-through of Verus and its approach to handling borrow termination. Finally, we thank Ralf Jung for many insightful remarks on an early version of this paper.

REFERENCES

2017. Not possible to bind to a pattern. <https://github.com/FStarLang/FStar/issues/1288>.
2021. StackOverflow Developer Survey. <https://insights.stackoverflow.com/survey/2021>.
2022. Verus, an Experimental Verification Framework for Rust-like code. <https://github.com/secure-foundations/verus/blob/004eadd8c31b60c886f7a8c8b568806e46a49a78/source/docs/design/returning-mutable-references.md>.
- Sidney Amani, Alex Hixon, Zilin Chen, Christine Rizkallah, Peter Chubb, Liam O'Connor, Joel Beeren, Yutaka Nagashima, Japheth Lim, Thomas Sewell, Joseph Tuong, Gabriele Keller, Toby Murray, Gerwin Klein, and Gernot Heiser. 2016. Cogent: Verifying High-Assurance File System Implementations. *SIGARCH Comput. Archit. News* 44, 2 (mar 2016), 175–188. <https://doi.org/10.1145/2980024.2872404>
- Qinxiang Cao Andrew W Appel, Lennart Beringer. 2021. Verifiable C. <https://softwarefoundations.cis.upenn.edu/vc-current/index.html>.
- V. Astrauskas, P. Müller, F. Poli, and A. J. Summers. 2019. Leveraging Rust Types for Modular Specification and Verification, In Object-Oriented Programming Systems, Languages, and Applications (OOPSLA). *Proc. ACM Program. Lang.* 3, OOPSLA, 147:1–147:30. <https://doi.org/10.1145/3360573>
- Thibaut Balabonski, François Pottier, and Jonathan Protzenko. 2016. The design and formalization of Mezzo, a permission-based programming language. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 38, 4 (2016), 1–94.
- Michael A. Bender, Martin Farach-Colton, William Jannen, Rob Johnson, Bradley C. Kuszmaul, Donald E. Porter, Jun Yuan, and Yang Zhan. 2015. An Introduction to Be-trees and Write-Optimization. *login Usenix Mag.* 40 (2015).
- Karthikeyan Bhargavan, Barry Bond, Antoine Delignat-Lavaud, Cédric Fournet, Chris Hawblitzel, Catalin Hritcu, Samin Ishtiaq, Markulf Kohlweiss, Rustan Leino, Jay Lorch, et al. 2017. Everest: Towards a verified, drop-in replacement of HTTPS. In *2nd Summit on Advances in Programming Languages (SNAPL 2017)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik.
- Aaron Bohannon, J Nathan Foster, Benjamin C Pierce, Alexandre Pilkiewicz, and Alan Schmitt. 2008. Boomerang: resourceful lenses for string data. In *Proceedings of the 35th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 407–419.
- John Boyland, James Noble, and William Retert. 2001. Capabilities for sharing. In *European Conference on Object-Oriented Programming*. Springer, 2–27.
- Qinxiang Cao, Lennart Beringer, Samuel Gruetter, Josiah Dodds, and Andrew W Appel. 2018. VST-Floyd: A separation logic tool to verify correctness of C programs. *Journal of Automated Reasoning* 61, 1 (2018), 367–422.
- Arthur Charguéraud and François Pottier. 2008. Functional translation of a calculus of capabilities. In *Proceedings of the 13th ACM SIGPLAN international conference on Functional programming*. 213–224.
- David G Clarke, John M Potter, and James Noble. 1998. Ownership types for flexible alias protection. In *Proceedings of the 13th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. 48–64.
- Xavier Denis, Jacques-Henri Jourdan, and Claude Marché. 2021. *The Creusot Environment for the Deductive Verification of Rust Programs*. Research Report RR-9448. Inria Saclay - Île de France. <https://hal.inria.fr/hal-03526634>
- Andrew Ferriaiuolo, Andrew Baumann, Chris Hawblitzel, and Bryan Parno. 2017. Komodo: Using verification to disentangle secure-enclave hardware from software. In *Proceedings of the 26th Symposium on Operating Systems Principles*. 287–305.
- Matthew Fluet, Greg Morrisett, and Amal Ahmed. 2006. Linear regions are all you need. In *European Symposium on Programming*. Springer, 7–21.
- Chris Hawblitzel, Jon Howell, Jacob R Lorch, Arjun Narayan, Bryan Parno, Danfeng Zhang, and Brian Zill. 2014. Ironclad Apps: {End-to-End} Security via Automated {Full-System} Verification. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. 165–181.
- Son Ho and Jonathan Protzenko. 2022a. Aeneas: A Verification Toolchain for Rust Programs. <https://github.com/sonmarcho/aeneas>.
- Son Ho and Jonathan Protzenko. 2022b. *Aeneas: Rust Verification by Functional Translation*. <https://doi.org/10.5281/zenodo.6672939>

- Son Ho and Jonathan Protzenko. 2022c. Aeneas: Rust Verification by Functional Translation (Long Version). <https://doi.org/10.48550/ARXIV.2206.07185>
- Son Ho, Jonathan Protzenko, Abhishek Bichhawat, and Karthikeyan Bhargavan. 2021. Noise*: A Library of Verified High-Performance Secure Channel Protocol Implementations. (2021).
- Graydon Hoare. 2022. https://twitter.com/graydon_pub/status/1492792051657629698.
- Uri Juhasz, Ioannis T Kassios, Peter Müller, Milos Novacek, Malte Schwerhoff, and Alexander J Summers. 2014. *Viper: A verification infrastructure for permission-based reasoning*. Technical Report. ETH Zurich.
- Ralf Jung, Hoang-Hai Dang, Jeehoon Kang, and Derek Dreyer. 2019. Stacked borrows: an aliasing model for Rust. *Proceedings of the ACM on Programming Languages* 4, POPL (2019), 1–32.
- Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. 2017. RustBelt: Securing the foundations of the Rust programming language. *Proceedings of the ACM on Programming Languages* 2, POPL (2017), 1–34.
- Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. 2018. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming* 28 (2018).
- Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, et al. 2009. seL4: Formal verification of an OS kernel. In *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*. 207–220.
- K Rustan M Leino. 2010. Dafny: An automatic program verifier for functional correctness. In *International Conference on Logic for Programming Artificial Intelligence and Reasoning*. Springer, 348–370.
- Jialin Li, Andrea Lattuada, Yi Zhou, Jonathan Cameron, Jon Howell, Bryan Parno, and Chris Hawblitzel. 2022. Linear Types for Large-Scale Systems Verification. In *Proceedings of the ACM Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*.
- Jacob R Lorch, Yixuan Chen, Manos Kapritsos, Bryan Parno, Shaz Qadeer, Upamanyu Sharma, James R Wilcox, and Xueyuan Zhao. 2020. Armada: low-effort verification of high-performance concurrent programs. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*. 197–210.
- Niko Matsakis. 2018. Regions are Sets of Loans. <http://smallcultfollowing.com/babysteps/blog/2018/04/27/an-alias-based-formulation-of-the-borrow-checker/>.
- Yusuke Matsushita, Xavier Denis, Jacques-Henri Jourdan, and Derek Dreyer. 2022. RustHorn-Belt: A semantic foundation for functional verification of Rust programs with unsafe code. In *Proceedings of the 43rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.
- Yusuke Matsushita, Takeshi Tsukada, and Naoki Kobayashi. 2020. RustHorn: CHC-Based Verification for Rust Programs.. In *ESOP*. 484–514.
- Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. 2015. The Lean theorem prover (system description). In *International Conference on Automated Deduction*. Springer, 378–388.
- LIAM O’Connor, Zilin Chen, Christine Rizkallah, Vincent Jackson, Sidney Amani, Gerwin Klein, Toby Murray, Thomas Sewell, and Gabriele Keller. 2021. Cogent: uniqueness types and certifying compilation. *Journal of Functional Programming* 31 (2021).
- François Pottier. 2017. Verifying a hash table and its iterators in higher-order separation logic. In *Proceedings of the 6th ACM SIGPLAN Conference on Certified Programs and Proofs*. 3–16.
- Jonathan Protzenko. 2014. *Mezzo: a typed language for safe effectful concurrent programs*. Ph. D. Dissertation. Université Paris Diderot-Paris 7.
- Jonathan Protzenko, Jean Karim Zinzindohoué, Aseem Rastogi, Tahina Ramananandro, Peng Wang, Santiago Zanella Béguelin, Antoine Delignat-Lavaud, Catalin Hritcu, Karthikeyan Bhargavan, Cédric Fournet, et al. 2017. Verified low-level programming embedded in F. *Proc. ACM program. lang.* 1, ICFP (2017), 17–1.
- John C Reynolds. 2002. Separation logic: A logic for shared mutable data structures. In *Proceedings 17th Annual IEEE Symposium on Logic in Computer Science*. IEEE, 55–74.
- Xavier Rival. 2011. Abstract Domains for the Static Analysis of Programs Manipulating Complex Data Structures. *Habilitation à diriger des recherches, École Normale Supérieure* (2011).
- Michael Sammler, Rodolphe Lepigre, Robbert Krebbers, Kayvan Memarian, Derek Dreyer, and Deepak Garg. 2021. RefinedC: automating the foundational verification of C code with refined ownership types. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. 158–174.
- The Rust Compiler Team. 2021. The Polonius Book. <https://rust-lang.github.io/polonius/>.
- The Rust Compiler Team. 2022. Guide to rustc development. https://rustc-dev-guide.rust-lang.org/borrow_check/two_phase_borrows.html.
- Mads Tofte and Jean-Pierre Talpin. 1997. Region-based memory management. *Information and computation* 132, 2 (1997), 109–176.
- Sebastian Ullrich. 2016. Simple verification of rust programs via functional purification. *Master’s Thesis, Karlsruher Institut für Technologie (KIT)* (2016).

- Philip Wadler. 1990. Linear types can change the world!. In *Programming concepts and methods*, Vol. 3. Citeseer, 5.
- Fabian Wolff, Aurel Bilý, Christoph Matheja, Peter Müller, and Alexander J. Summers. 2021. Modular Specification and Verification of Closures in Rust. *Proc. ACM Program. Lang.* 5, OOPSLA, Article 145 (oct 2021), 29 pages. <https://doi.org/10.1145/3485522>